

Adaptive Coordinate Random Search for Hyperparameter Optimization

Dennis Diehl

June 11, 2026



Johannes Gutenberg University Mainz
FB08
Institute of Computer Science
Data Mining

Bachelor thesis
submitted for the award of the degree
Bachelor of Science (B.Sc.)

Adaptive Coordinate Random Search for Hyperparameter Optimization

Dennis Diehl

- | | |
|--------------------|--|
| <i>1. Reviewer</i> | Professor Dr. Stefan Kramer
Department of Data Mining
Johannes Gutenberg University Mainz |
| <i>2. Reviewer</i> | Dr. Markus Blumenstock
Department of Theoretical Computer Science
Johannes Gutenberg University Mainz |
| <i>Supervisor</i> | Kirsten Köbschall |

June 11, 2026

Dennis Diehl

Adaptive Coordinate Random Search for Hyperparameter Optimization

Bachelor thesis, June 11, 2026

Reviewers: Professor Dr. Stefan Kramer and Dr. Markus Blumenstock

Supervisor: Kirsten Köbschall

Johannes Gutenberg University Mainz

Data Mining

Institute of Computer Science

FB08

Staudingerweg 9

55128 Mainz

Abstract

Hyperparameter optimization (HPO) is essential for the predictive performance of machine learning models, but the optimization process itself consumes computational resources and energy, an effect Bayesian Optimization (BO) via the Tree-Structured Parzen Estimator (TPE) compounds by refitting its surrogate model after every trial. This thesis investigates whether a surrogate-free, coordinatewise search that adaptively weights hyperparameters by their contribution to performance can match or exceed BO's quality at lower cost, and proposes Adaptive Coordinate Random Search (ACRS) to address this question. Benchmarked against BO, Random Search (RS), and CMA-ES on a Multilayer Perceptron (MLP) and a Random Forest (RF) across multiple tabular binary classification datasets and seeds, ACRS matches or exceeds BO's predictive quality for both models, with differences within the across-seed standard deviation, while reducing runtime and energy consumption by roughly 10–27% relative to BO, depending on model type. ACRS thus offers a resource-efficient, surrogate-free alternative to BO.

Abstract (German)

Hyperparameter-Optimierung (HPO) ist entscheidend für die Vorhersagequalität von Machine-Learning-Modellen, doch der Optimierungsprozess selbst verbraucht Rechenressourcen und Energie. Bayessche Optimierung (BO) mittels Tree-Structured Parzen Estimator (TPE) verstärkt diesen Effekt zusätzlich, da ihr Surrogatmodell nach jedem Trial neu angepasst werden muss. Diese Arbeit untersucht, ob eine surrogatfreie, koordinatenweise Suche die Qualität von BO bei geringeren Kosten erreichen oder übertreffen kann, wenn sie Hyperparameter adaptiv nach ihrem Beitrag zur Modellgüte gewichtet. Dazu stellt sie Adaptive Coordinate Random Search (ACRS) vor. ACRS wird mit BO, Random Search (RS) und CMA-ES auf einem Multilayer Perceptron (MLP) und einem Random Forest (RF) über mehrere tabellarische binäre Klassifikationsdatensätze und Seeds verglichen. Dabei erreicht oder übertrifft ACRS die Vorhersagequalität von BO für beide Modelle, wobei alle Unterschiede innerhalb des seedbedingten Rauschens liegen. Gleichzeitig reduziert ACRS Laufzeit und Energieverbrauch je nach Modelltyp um etwa 10–27% gegenüber BO. ACRS bietet damit eine ressourceneffiziente, surrogatfreie Alternative zu BO.

Contents

1. Introduction	1
2. Related Work	3
2.1. The HPO Landscape	3
2.2. Surrogate-Free Methods	3
2.3. Surrogate-Based Methods	4
2.4. Coordinatewise and Adaptive Search Strategies	5
2.5. Comparative Studies and Research Gap	6
3. Preliminaries	7
3.1. Hyperparameter Formalization	7
3.2. The Hyperparameter Optimization Problem	7
3.3. Evaluation Metrics	8
3.4. Models	9
4. Methodology	13
4.1. Hyperparameter Search Spaces	13
4.2. Optimization Methods	14
4.2.1. Adaptive Coordinate Random Search (ACRS)	15
4.2.2. Bayesian Optimization via TPE (BO)	19
4.2.3. Baseline Methods	20
4.3. Complexity Analysis	21
5. Benchmark Setup	23
5.1. Benchmark Datasets	23
5.2. Benchmark Design	24
6. Results	27
6.1. ACRS Ablation Study	27
6.2. Model Quality	28
6.3. Convergence Behavior	30
6.4. Resource Efficiency	32
7. Discussion	33
7.1. Interpretation of Model Quality	33
7.2. Efficiency Advantage and Its Mechanism	34
7.3. Ablation: Effect of the ACRS Design Choices	35
7.4. Limitations	36
8. Conclusion	39
8.1. Summary of Findings	39
8.2. Practical Implications and Future Work	40

Bibliography	41
List of Figures	43
List of Tables	45
A. Appendix	47
A.1. Bayesian Optimization Algorithm	47
A.2. Performance Distribution	48
A.3. Convergence Curves for All Datasets	48
A.4. Runtime Table	49

Introduction

Hyperparameter optimization (HPO) is a necessary step when training machine learning models, since the choice of hyperparameters such as learning rate, network architecture, or regularization strength substantially affects the resulting model's predictive performance. The optimization process itself, however, consumes computational resources and energy, an effect that scales with both the number of trials evaluated and the cost of each individual trial. Bayesian Optimization (BO) via the Tree-Structured Parzen Estimator (TPE) is the most established systematic approach to HPO, guiding the search by fitting a probabilistic surrogate model of the objective function and refitting it after every trial. This refitting adds a computational cost on top of the cost of training and evaluating the model itself, a cost that accumulates over the full search and that weighs more heavily the cheaper each individual trial is.

For small to medium-sized tabular classification tasks, a common setting in practice, this raises the question of whether the additional cost of fitting and refitting a surrogate model is justified, or whether simpler surrogate-free strategies can reach comparable model quality at lower cost. Theoretical results suggest that surrogate-free strategies can indeed be competitive. Hutter et al. [15] show that performance variation in HPO is typically dominated by a small number of hyperparameters, and Nutini et al. [20] prove that prioritizing the most impactful coordinate converges no slower than random coordinate selection. Building on this, Florea and Andonie [11] show that importance-weighted random search reaches the global optimum with a probability at least as high as uniform random search. Together, these results motivate a coordinatewise, adaptively weighted, surrogate-free search strategy as a credible alternative to BO. This combination has, however, not been systematically benchmarked against BO under controlled conditions that also track its resource cost.

Two gaps in the existing literature motivate this thesis. First, few studies compare BO against lightweight, surrogate-free, coordinatewise strategies that adaptively prioritize hyperparameters, leaving open how such a method performs relative to an established surrogate-based baseline. Second, the energy consumption of the optimization process itself is rarely measured, even though tools such as CodeCarbon [8] have made this feasible. Closing these gaps requires such a method, evaluated against BO under matched conditions that also track resource cost.

This thesis addresses both gaps by proposing Adaptive Coordinate Random Search (ACRS), a surrogate-free HPO method that optimizes one hyperparameter at a time while holding the others fixed at their current best values, and that maintains adaptive weights prioritizing hyperparameters by their

observed contribution to performance. Two optional extensions, range shrinking and Gaussian candidate sampling, contract the search around the current best value, yielding additional ACRS variants explored in this thesis. The contribution of this thesis is twofold. It introduces the design of ACRS, and it provides a controlled benchmark of ACRS against BO (TPE) as the main surrogate-based competitor, with Random Search (RS) and Covariance Matrix Adaptation Evolution Strategy (CMA-ES) as additional surrogate-free baselines. The benchmark is guided by two questions. *Does ACRS match or exceed BO's model quality? And does it do so at lower computational and energy cost?*

The benchmark compares BO, the ACRS variants, and the surrogate-free baselines RS and CMA-ES on two model types, a Multilayer Perceptron (MLP) and a Random Forest (RF), across multiple tabular binary classification datasets and repeated random seeds. The evaluation budget is shared across all methods and derived from the structure of the ACRS algorithm, ensuring a fair comparison. The comparison covers final predictive performance, measured as Area Under the Receiver Operating Characteristic Curve (AUROC) on the held-out test set, convergence behavior measured against both trial number and cumulative training time, and total runtime and energy consumption measured with CodeCarbon [8].

This thesis is structured as follows: In **Chapter 2**, the HPO landscape is reviewed and ACRS is positioned relative to existing surrogate-free and surrogate-based methods, followed by the necessary background on the MLP and RF classifiers and the optimization methods compared in this thesis in **Chapter 3**. The ACRS algorithm and its variants are then presented in **Chapter 4**, alongside the formal definitions of BO, RS, and CMA-ES and the search spaces used. Before the results are reported, the benchmark datasets and experimental design are described in **Chapter 5**. **Chapter 6** reports the results, and **Chapter 7** interprets them and discusses the limitations of this scope. Finally, in **Chapter 8**, the findings and their practical implications are summarized.

Related Work

This chapter surveys the HPO landscape to position ACRS, the method proposed in this thesis, against existing approaches. It covers surrogate-free methods (Grid Search, RS, and CMA-ES), surrogate-based BO via TPE, and the coordinatewise and adaptive search strategies from which ACRS draws its design. The survey concludes by identifying the research gap that ACRS addresses.

2.1 The HPO Landscape

HPO aims to find a configuration of hyperparameters that maximizes the generalization performance of a machine learning model. A broad taxonomy of HPO methods has emerged over the past two decades, organized along two principal axes, namely whether the method maintains a surrogate model of the objective function and how it selects candidate configurations [10, 28].

Surrogate-free methods evaluate configurations without modeling the relationship between hyperparameters and performance. They range from exhaustive strategies such as Grid Search to randomized methods such as RS and evolutionary strategies such as CMA-ES. In contrast, surrogate-based methods construct a probabilistic model that approximates the mapping from hyperparameters to performance and use it to propose promising candidates. BO is the most prominent representative of this family, fitting a surrogate and using an acquisition function to balance exploration and exploitation [21].

Beyond these core families, bandit-based methods such as Hyperband [18] allocate budgets adaptively by terminating poorly performing configurations early, while Population-Based Training mutates and recombines configurations across generations. These methods are outside the scope of this thesis but are mentioned here for completeness.

2.2 Surrogate-Free Methods

Surrogate-free methods require no probabilistic model of the objective and are therefore easy to implement, trivially parallelizable, and applicable to any search space. The three methods discussed here represent distinct design points within this family and collectively motivate the ACRS strategy developed in this thesis.

Grid Search exhaustively evaluates every point in the Cartesian product of predefined candidate sets. While straightforward and fully reproducible, it suffers from the curse of dimensionality and wastes evaluations on dimensions that have little impact on performance [10, 28]. ACRS adopts its coordinatewise evaluation principle of varying one dimension at a time while holding the others fixed.

A more sample-efficient alternative is RS. Bergstra and Bengio [5] showed that drawing configurations independently and uniformly at random substantially outperforms Grid Search when only a few hyperparameters strongly influence performance, because random sampling covers each dimension independently and avoids wasting evaluations on irrelevant combinations. RS requires no memory of past evaluations and is trivially parallelizable, but by construction it cannot concentrate sampling in promising regions of the search space, since every configuration is drawn without regard to previous results. In this thesis it serves as a memoryless surrogate-free baseline, and its uniform sampling principle is adopted within each coordinate sweep of ACRS. The formal description is given in Chapter 4.

A more adaptive surrogate-free approach is CMA-ES [13], a population-based evolution strategy that maintains a multivariate Gaussian distribution over the search space and updates it based on the performance of sampled populations [12]. At each generation, a population of candidate configurations is drawn from the current distribution, evaluated on the objective, and used to update both the mean and the covariance matrix of the distribution. The covariance adaptation allows CMA-ES to capture dependencies between hyperparameters and progressively focus sampling toward high-performing regions, without maintaining an explicit surrogate model of the objective. Unlike RS, CMA-ES thus adapts its sampling distribution over time, making it a stronger adaptive baseline. In this thesis, CMA-ES serves as an adaptive surrogate-free baseline to isolate the contribution of ACRS’s coordinatewise structure from what can be achieved by any adaptive sampling strategy. The formal description is given in Chapter 4.

2.3 Surrogate-Based Methods

Surrogate-based methods address the sample inefficiency of surrogate-free approaches by maintaining a probabilistic model of the objective function and using it to propose configurations that are likely to improve over the current best [21]. The canonical instantiation of BO uses a Gaussian Process (GP) as the surrogate and Expected Improvement as the acquisition function [23]. GP-based BO provides calibrated uncertainty estimates but scales cubically with the number of observations, limiting its applicability for large evaluation budgets.

TPE [4] addresses this limitation by replacing the GP with a density-ratio surrogate that scales more favorably and handles categorical and conditional parameters naturally. A third variant, SMAC [16], uses a Random Forest

surrogate for robustness in high-dimensional and mixed configuration spaces. The detailed mechanism of TPE is presented in Chapter 4 (Algorithm 3).

This thesis employs TPE as the surrogate-based baseline because it natively supports the mixed search spaces used in the experiments, scales well under the moderate evaluation budget, and is efficiently implemented in Optuna [1].

2.4 Coordinatewise and Adaptive Search Strategies

The ACRS strategy draws on a line of work that combines coordinatewise evaluation with adaptive mechanisms for prioritizing and focusing the search. Coordinate descent optimizes one variable at a time while holding the others fixed [27]. Applied to HPO and combined with random sampling per dimension, this reduces the evaluation cost compared to a full grid while preserving the geometric advantage of random sampling identified by Bergstra and Bengio [5].

The importance of individual hyperparameters provides a further motivation for coordinatewise strategies. Hutter et al. [15] show empirically that even in high-dimensional configuration spaces the bulk of performance variation is attributable to just a few hyperparameters, motivating importance-weighted search. Nutini et al. [20] provide a theoretical counterpart, proving that prioritizing the most impactful coordinate converges faster than random coordinate selection. Building on these results, Florea and Andonie [11] propose Weighted Random Search, which assigns sampling probabilities proportional to estimated hyperparameter importance and proves that its probability of reaching the global optimum is at least as high as that of standard RS. Weighted Random Search concentrates sampling on more influential dimensions, but still draws complete configurations by varying all hyperparameters simultaneously with adjusted probabilities. It does not fix other hyperparameters at their current best values during a step, and it does not narrow the search range around promising regions over time.

ACRS extends Weighted Random Search in two directions. First, it introduces a coordinatewise structure, evaluating one hyperparameter at a time while holding all others fixed at the current best rather than drawing complete configurations, which reduces individual trial cost in settings where that cost is heterogeneous. Second, it optionally contracts the search range around the current best value after each coordinate step through range shrinking, progressively focusing the search beyond what reweighting alone achieves. The formal definition and pseudocode are presented in Chapter 4 (Algorithm 1).

2.5 Comparative Studies and Research Gap

A number of studies have compared HPO methods empirically. Yang and Shami [28] survey both theory and practice of HPO algorithms and find that the relative ranking of methods varies across tasks and budgets, echoing the No Free Lunch theorem [26]. Feurer and Hutter [10] provide a comprehensive review of HPO within the broader AutoML landscape, noting that surrogate-based methods generally outperform surrogate-free baselines given sufficient budget, but that the margin depends heavily on the dimensionality and structure of the search space. Li et al. [18] propose Hyperband and demonstrate competitive performance with significantly fewer resources on several benchmarks.

Despite this body of work, several gaps remain. Few studies systematically compare BO against lightweight, surrogate-free coordinatewise strategies that adaptively prioritize hyperparameters and shrink their search ranges. The energy cost of the optimization process itself is rarely measured; tools such as CodeCarbon [8] have made carbon tracking feasible, but most HPO benchmarks still omit resource consumption metrics. Small tabular datasets are also underrepresented in the HPO literature, which tends to focus on large-scale deep learning benchmarks. Finally, seed variability across repeated runs is seldom reported explicitly, making it difficult to distinguish genuine algorithmic differences from random fluctuations.

The present work addresses these gaps by benchmarking TPE-based BO against ACRS, with RS and CMA-ES as additional baselines, across five tabular datasets, using ten random seeds per configuration and tracking both predictive performance and energy consumption. The formal definitions of the optimization algorithms, search spaces, and evaluation protocol are presented in the following chapters.

Preliminaries

Every rigorous comparison rests on a common formal language. The following sections establish this foundation, formalizing the HPO problem, introducing the metrics used to assess predictive quality and computational cost, and describing the two model architectures whose hyperparameters are optimized in the experiments. The concrete search spaces and optimization algorithms are presented in Chapter 4.

3.1 Hyperparameter Formalization

Hyperparameters are design choices that govern the structure or training procedure of a machine learning model and must be fixed before training begins. Unlike model parameters θ (e.g. neural network weights or decision tree split points), which are learned automatically from data, hyperparameters are not updated during training. Examples include the learning rate, the number of hidden layers, or the splitting criterion of a decision tree [10].

Formally, consider a learning algorithm A that, given a training dataset D_{train} and a hyperparameter configuration λ , produces a trained model $A(D_{\text{train}}, \lambda)$. The configuration λ is a vector of n components, where each component λ_i is the value assigned to the i -th hyperparameter, drawn from its domain Λ_i . A domain may be continuous ($\Lambda_i \subseteq \mathbb{R}$), integer-valued ($\Lambda_i \subseteq \mathbb{Z}$), or categorical ($\Lambda_i = \{v_1, \dots, v_k\}$). The full configuration space is the Cartesian product

$$\Lambda = \Lambda_1 \times \Lambda_2 \times \dots \times \Lambda_n, \quad (3.1)$$

and a concrete configuration is a vector $\lambda = (\lambda_1, \dots, \lambda_n) \in \Lambda$. In this thesis, the MLP search space comprises $n = 8$ hyperparameters and the RF search space $n = 7$ hyperparameters; the concrete domains are defined in Chapter 4.

3.2 The Hyperparameter Optimization Problem

The goal of HPO is to find the configuration that maximizes a black-box objective function $f: \Lambda \rightarrow \mathbb{R}$, given by

$$\lambda^* = \arg \max_{\lambda \in \Lambda} f(\lambda). \quad (3.2)$$

In this thesis, $f(\lambda) := \mathcal{V}(A(D_{\text{train}}, \lambda), D_{\text{val}})$ denotes the validation Area Under the Receiver Operating Characteristic Curve (AUROC) of the model trained with configuration λ on D_{train} and evaluated on D_{val} (defined in Section 3.3).

Three properties make this optimization problem challenging. First, the objective is a black box, meaning no gradient of f with respect to λ is available, because f depends on the entire training procedure. Second, each evaluation is expensive, as it requires a complete training run of the underlying model. Third, the No Free Lunch theorem [26] implies that no single optimization strategy dominates all others across all possible objective landscapes.

3.3 Evaluation Metrics

To assess hyperparameter configurations, this thesis reports AUROC and Accuracy as measures of predictive quality, and Training Time and Energy Consumption as measures of resource efficiency.

AUROC [9] measures how well a classifier distinguishes between positive and negative instances across all possible classification thresholds, or equivalently, the probability that a randomly chosen positive instance will be ranked higher than a randomly chosen negative instance. By assigning a probability score to each input and continuously varying the decision threshold from its maximum to its minimum value, the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR) is traced, forming the Receiver Operating Characteristic (ROC) curve. These rates are defined as

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}, \quad (3.3)$$

where TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives. The AUROC is then defined as the area under the ROC curve,

$$\text{AUROC} = \int_0^1 \text{TPR}(f) df, \quad (3.4)$$

where $f \in [0, 1]$ denotes the FPR value and $\text{TPR}(f)$ the corresponding TPR value on the ROC curve. A value of 0.5 corresponds to a random classifier, while 1.0 indicates perfect separation. Because AUROC is threshold-independent and robust to class imbalance, it is used as the primary metric in this thesis.

Accuracy measures the fraction of instances for which the predicted class label matches the true label. Using the same notation as above,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (3.5)$$

A value of 1.0 corresponds to perfect classification, while 0.0 means every instance is misclassified. Unlike AUROC, Accuracy depends on a fixed decision threshold and can be misleading under class imbalance.

Training time captures the duration of a single trial, i.e. one complete training run with a fixed hyperparameter configuration. It provides a hardware-dependent but practically relevant measure of computational cost.

Energy consumption is recorded in kilowatt-hours using CodeCarbon [8]. It measures the total electrical energy drawn by the hardware over the duration of a run, approximated by periodically sampling CPU and GPU power counters¹.

3.4 Models

This thesis employs two model families, an MLP and an RF, that represent fundamentally different learning paradigms. Their distinct architectures and training mechanisms give rise to different hyperparameter spaces, making them well-suited for comparing optimization strategies across diverse settings. Both are applied to binary classification tasks ($C = 2$) throughout.

Multilayer Perceptron An MLP [3] is a feedforward neural network that maps an input $x \in \mathbb{R}^d$, where d is the number of input features, to a probability distribution over C target classes. It consists of an input layer, L hidden layers, and an output layer, where data flows in one direction from input to output.

Each hidden layer $l \in \{1, \dots, L\}$ transforms its input via a weighted sum followed by a nonlinear activation function. Denoting the output of layer l as $h^{(l)}$, with $h^{(0)} = x$, this computation is

$$h^{(l)} = \sigma\left(W^{(l)} h^{(l-1)} + b^{(l)}\right), \quad (3.6)$$

where $W^{(l)}$ and $b^{(l)}$ are the learnable weight matrix and bias vector of layer l , and σ is an element-wise nonlinear activation function. Without this nonlinearity, stacking multiple layers would be mathematically equivalent to a single linear transformation, limiting the network to linear decision boundaries. This thesis considers ReLU and tanh as activation choices, defined as

$$\text{ReLU}(x) = \max(0, x), \quad \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (3.7)$$

The output layer applies a linear transformation to the last hidden state,

$$z = W^{(\text{out})} h^{(L)} + b^{(\text{out})}, \quad (3.8)$$

and then a softmax function to produce a probability distribution $\hat{p} \in \mathbb{R}^C$ over the C classes,

$$\hat{p}_c = \frac{\exp(z_c)}{\sum_{c'=1}^C \exp(z_{c'})}, \quad (3.9)$$

¹Since this thesis runs on CPU-only hardware, GPU energy is not measured (see Section 5.2).

where z_c is the c -th entry of the logit vector $z \in \mathbb{R}^C$. The predicted class is then

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} \hat{p}_c. \quad (3.10)$$

The network is trained by minimizing the cross-entropy loss between the predicted probabilities and the true labels,

$$\mathcal{L}(\hat{p}, y) = - \sum_{c=1}^C y_c \log \hat{p}_c, \quad (3.11)$$

where $y_c \in \{0, 1\}$ is the true label indicator for class c . The loss is minimized by a gradient-based optimizer that iteratively adjusts all weights $W^{(l)}$ and biases $b^{(l)}$. This thesis considers Adam [17], SGD, and AdamW as optimizer choices. Training proceeds over multiple passes through the training set; one complete pass is called an epoch.

A common problem during training is overfitting, where the network memorizes noise in the training data and fails to generalize to unseen examples. Two mechanisms counteract this. Weight decay penalizes large weights, encouraging simpler solutions. Dropout [24] randomly deactivates a fraction of neurons during each training step, preventing the network from relying too heavily on individual neurons.

The hyperparameters of an MLP govern the network architecture, training procedure, and regularization, including the number of layers, hidden dimensionality, learning rate, optimizer, dropout rate, and weight decay. Their concrete domains are defined in Chapter 4.

Random Forest An RF [7] is an ensemble of T decision trees $\mathcal{F} = \{h_1, h_2, \dots, h_T\}$, where each tree h_t is trained independently. At prediction time, the input $x \in \mathbb{R}^d$ is passed to every tree h_t , each of which produces a class probability estimate. These per-tree estimates are then averaged to form the ensemble prediction, yielding a more robust and accurate result than any individual tree alone.

Each tree h_t is trained by recursively partitioning the feature space to classify inputs. At each internal node v , h_t selects the feature and threshold that best separate the training samples by class, aiming to produce child nodes that are as pure as possible. Purity is measured by an impurity criterion; this thesis uses Gini impurity and entropy [3] as selectable criteria. Denoting the proportion of samples belonging to class c at node v as $\pi_c(v)$, where $c \in \{1, \dots, C\}$, the Gini impurity $G(v)$ and entropy $H(v)$ are defined as

$$G(v) = 1 - \sum_{c=1}^C \pi_c(v)^2, \quad H(v) = - \sum_{c=1}^C \pi_c(v) \log_2 \pi_c(v). \quad (3.12)$$

Splitting of h_t continues until a stopping condition is reached, such as a maximum tree depth or a minimum number of samples per leaf.

A single deep tree tends to overfit, memorizing noise in the training data that does not generalize to unseen examples. The RF addresses this through two sources of randomness that ensure the trees are diverse. First, each tree h_t is trained via bagging [6], i.e. on a bootstrap sample drawn with replacement from the full training set. Second, at each split, only a random subset of features is considered as candidates, further decorrelating the trees.

Formally, each tree h_t outputs a per-tree class probability estimate $p_{t,c}(x)$ for each class c , and the ensemble probability $\hat{p}_c(x)$ is defined as

$$\hat{p}_c(x) = \frac{1}{T} \sum_{t=1}^T p_{t,c}(x), \quad p_{t,c}(x) = \frac{n_{t,c}(x)}{n_t(x)}, \quad (3.13)$$

where $n_{t,c}(x)$ and $n_t(x)$ denote the number of class- c samples and total samples in the leaf of h_t reached by x . The predicted class is then

$$\hat{y} = \arg \max_{c \in \{1, \dots, C\}} \hat{p}_c(x). \quad (3.14)$$

The hyperparameters of the RF govern both the individual trees and the ensemble as a whole, controlling aspects such as tree depth, ensemble size, and the degree of randomness introduced during training. Their concrete domains are defined in Chapter 4.

Methodology

This chapter defines the methodological components of the benchmark. The search spaces for both models are established first, as they form the domain over which all optimization methods operate. The optimization methods are then described in detail, with ACRS as the proposed method, BO via TPE as the main model-based competitor, and RS and CMA-ES as surrogate-free baselines. A complexity analysis concludes the chapter by examining how the evaluation budget of each method scales with the number of hyperparameters. The concrete experimental parameters, datasets, and evaluation protocol are specified in Chapter 5.

4.1 Hyperparameter Search Spaces

The search spaces for both models were designed to cover a broad range of plausible configurations while keeping the number of hyperparameters manageable. For each hyperparameter, the distribution and scale were chosen to reflect its domain structure. Hyperparameters that span multiple orders of magnitude are sampled on a logarithmic scale to give equal coverage to each decade. Bounded integer-valued hyperparameters are sampled uniformly. Discrete, unordered choices are treated as categorical variables. The resulting search spaces are summarized in Tables 4.1 and 4.2.

Tab. 4.1.: This table specifies the hyperparameter search space of the MLP.

Hyperparameter	Distribution	Range / Set	Scale
learning_rate	Log-Uniform	$[10^{-5}, 10^{-1}]$	logarithmic
batch_size	Uniform Integer	$2^e, e \in [4, 7]$	linear
hidden_dim	Uniform Integer	$[32, 256]$	linear
dropout	Uniform	$[0.0, 0.5]$	linear
num_layers	Categorical	$\{1, 2, 3\}$	–
optimizer_name	Categorical	$\{\text{adam}, \text{sgd}, \text{adamw}\}$	–
weight_decay	Log-Uniform	$[10^{-6}, 10^{-2}]$	logarithmic
activation	Categorical	$\{\text{relu}, \text{tanh}\}$	–

The learning rate and weight decay are sampled log-uniformly, as their effective range spans several orders of magnitude and a linear scale would allocate the vast majority of samples to large values. Batch size is parameterized as an integer exponent $e \in [4, 7]$, yielding effective values of $\{16, 32, 64, 128\}$; this encoding allows optimizers to treat the variable as a continuous integer rather

than a nominal category. Hidden dimension and dropout are sampled uniformly over their respective intervals, reflecting the absence of a natural scale preference. The remaining hyperparameters - number of layers, optimizer, and activation function - are categorical, as they represent discrete structural choices with no meaningful ordering.

Tab. 4.2.: This table specifies the hyperparameter search space of the RF.

Hyperparameter	Distribution	Range / Set	Scale
n_estimators	Uniform Integer	[50, 500]	linear
max_depth	Uniform Integer	[3, 30]	linear
min_samples_split	Uniform Integer	[2, 20]	linear
max_features	Categorical	{sqrt, log2}	–
min_samples_leaf	Uniform Integer	[1, 10]	linear
criterion	Categorical	{gini, entropy}	–
max_samples	Uniform	[0.5, 1.0]	linear

All integer and continuous hyperparameters are sampled uniformly, as none of their ranges span multiple orders of magnitude. The feature sampling strategy and split criterion are treated as categorical variables, as they represent discrete, unordered choices with no natural ordering.

4.2 Optimization Methods

This section describes the optimization methods evaluated in this work. The selection is motivated by the goal of covering a representative spectrum of search strategies. ACRS and its variants constitute the proposed method and form the primary subject of investigation. BO via TPE serves as the main model-based competitor and represents a widely used state-of-the-art method for sequential HPO. RS provides a memoryless baseline that makes no use of past observations, and CMA-ES represents a population-based, gradient-free evolutionary strategy. Together, these methods span the range from fully model-free to model-based search, enabling a meaningful comparison across fundamentally different approaches. To ensure a fair comparison, all methods use the same evaluation budget T .

4.2.1 Adaptive Coordinate Random Search (ACRS)

ACRS is a coordinate-wise optimization strategy that combines two core ideas. First, it optimizes one hyperparameter at a time while holding all others fixed, a principle known as *coordinate-wise optimization*, analogous to coordinate descent in continuous optimization. Second, it maintains *adaptive weights* that track how much improvement each hyperparameter has contributed. Empirically, performance variation is typically dominated by few hyperparameters even in high-dimensional spaces [15]. Theoretically, updating the coordinate with the largest observed gain converges no slower than random coordinate selection [20]. Assigning per-dimension sampling probabilities proportional to observed importance, an idea formalized for random search by Florea and Andonie [11], is the basis for the priority ordering used here. As introduced in Section 2.4, Weighted Random Search assigns these probabilities globally and still draws complete configurations, whereas ACRS evaluates one coordinate at a time with all others fixed at the current best, and optionally narrows the search range around the identified optimum after each step. ACRS is developed as a lightweight, model-free alternative to surrogate-based methods, directly addressing the research gap identified in Section 2.5.

ACRS operates on n hyperparameters and runs for R rounds, drawing L candidate values per hyperparameter in each coordinate step. Each hyperparameter j is associated with an adaptive weight w_j , reflecting its observed contribution to performance improvement, and a current search range $\mathcal{R}_j$, initialized to the full domain Λ_j . The per-step improvement Δ_j records the gain in objective value achieved during the coordinate step for hyperparameter j . When range shrinking is enabled, the shrinking factor $\alpha \in (0, 1)$ controls how strongly $\mathcal{R}_j$ is contracted around the current best value after each step.

Algorithm 1 details the procedure. The two boolean flags *use_normal* and *shrink* select between the four variants summarized in Table 4.3.

Algorithm 1 Adaptive Coordinate Random Search (ACRS)

Require: Search space Λ with n hyperparameters, objective f , samples per parameter L , rounds R , shrinking factor $\alpha \in (0, 1)$, flags *use_normal*, *shrink* $\in \{\text{true}, \text{false}\}$

```
1:  $\lambda^* \leftarrow \text{Random}(\Lambda)$ ;  $y^* \leftarrow f(\lambda^*)$ 
2:  $w_j \leftarrow 1/n$  for all  $j = 1, \dots, n$ 
3:  $\mathcal{R}_j \leftarrow \Lambda_j$  for all  $j = 1, \dots, n$ 
4: for  $r = 1$  to  $R$  do
5:    $\Delta_j \leftarrow 0$  for all  $j = 1, \dots, n$ 
6:    $\pi \leftarrow \text{PriorityOrder}(\{1, \dots, n\}, \mathbf{w})$  ▷ see Alg. 2
7:   for  $j \in \pi$  do
8:      $y_{\text{before}} \leftarrow y^*$  ▷ record  $y^*$  before step to measure improvement  $\Delta_j$ 
9:     for  $l = 1$  to  $L$  do
10:       $\lambda' \leftarrow \lambda^*$ 
11:       $\lambda'_j \leftarrow \text{Sample}(\mathcal{R}_j, \lambda_j^*, \text{use\_normal})$ 
12:       $y' \leftarrow f(\lambda')$ 
13:      if  $y' > y^*$  then
14:         $y^* \leftarrow y'$ ;  $\lambda^* \leftarrow \lambda'$ 
15:      end if
16:    end for
17:     $\Delta_j \leftarrow y^* - y_{\text{before}}$ 
18:    if shrink then
19:       $\mathcal{R}_j \leftarrow \text{Shrink}(\mathcal{R}_j, \lambda_j^*, \alpha)$  ▷ see Fig. 4.1
20:    end if
21:  end for
22:   $\text{total} \leftarrow \sum_{j=1}^n \Delta_j$ 
23:  if  $\text{total} > 0$  then
24:     $w_j \leftarrow \Delta_j / \text{total}$  for all  $j$ 
25:  else ▷ fallback: no improvement in this round
26:     $w_j \leftarrow 1/n$  for all  $j$ 
27:  end if
28: end for
29: return  $\lambda^*, y^*$ 
```

Lines 1–3 initialize the algorithm. A random configuration is drawn and evaluated as the initial best, the adaptive weights are set uniformly to $1/n$, and the search ranges are set to the full domain of each hyperparameter.

The outer loop (lines 4–28) iterates over R rounds. A single pass through all hyperparameters is generally insufficient, since each coordinate step fixes all other parameters at their current best values and the optimal setting for an already-visited parameter may shift as later parameters are updated. Repeating the process over multiple rounds lets the algorithm revisit each hyperparameter under progressively refined conditions and correct decisions that were suboptimal in earlier passes. At the start of each round, the per-parameter improvements Δ_j are reset to zero (line 5) and the processing order π is determined from the current weights $\mathbf{w}$ (line 6), so that hyperparameters with higher weights tend to appear earlier.

The inner coordinate loop (lines 7–21) processes each hyperparameter j in the order given by π . For each hyperparameter, L candidate configurations are evaluated (lines 9–16). Each candidate copies the current best configuration and replaces only component j with a value drawn via $\text{Sample}(\mathcal{R}_j, \lambda_j^*, \text{use_normal})$ (line 11). Sample draws uniformly from $\mathcal{R}_j$ if use_normal is false, and from $\mathcal{N}(\lambda_j^*, \sigma_j)$ clipped to $\mathcal{R}_j$ with $\sigma_j = |\mathcal{R}_j|/4$ if true. Categorical hyperparameters are always sampled uniformly. If a candidate improves upon the current best, λ^* and y^* are updated (lines 13–14). After all L samples, the improvement $\Delta_j = y^* - y_{\text{before}}$ is recorded (line 17), and if *shrink* is enabled, $\mathcal{R}_j$ is contracted around λ_j^* (lines 18–20). For a continuous range $[a_j, b_j]$, the updated range is

$$\mathcal{R}_j \leftarrow [\lambda_j^* - \alpha(\lambda_j^* - a_j), \lambda_j^* + \alpha(b_j - \lambda_j^*)], \quad (4.1)$$

clipped to the original bounds $[a_j, b_j]$. Integer-valued hyperparameters are treated analogously to continuous ones, with the contracted bounds rounded to the nearest integers and clipped to the original domain. Categorical ranges are left unchanged.

After each full pass, the total improvement is computed (line 22) and used to update the weights (lines 23–27). Each w_j is set proportionally to Δ_j if any improvement was observed, and reset to uniform otherwise. This ensures that hyperparameters which consistently drive performance gains receive increasing attention in subsequent rounds.

Figure 4.1 brings these steps together visually, using the MLP search space from Table 4.1 as a concrete example, where the five continuous hyperparameters occupy the upper block and the three categorical ones the lower block. For each hyperparameter, the highlighted segment marks the current search range $\mathcal{R}_j$ and the dot the value λ_j^* selected in that round. Across the rounds the chosen value is refined and, when shrinking is enabled, the range contracts around it following Equation 4.1, whereas categorical ranges stay fixed.

ACRS: how each hyperparameter is sampled and progressively narrowed

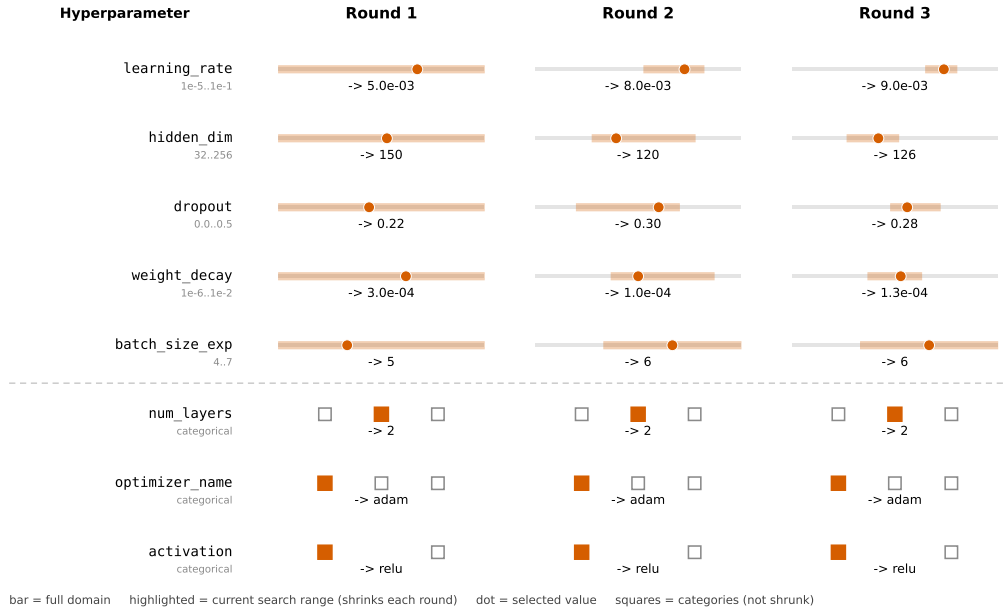


Fig. 4.1.: Schematic illustration of the ACRS mechanism on the MLP search space. Each row is a hyperparameter and each column a round. For a continuous hyperparameter, the grey bar spans its full domain, the highlighted segment marks the current search range, and the dot marks the value selected in that round. From left to right the highlighted range contracts around the selected value, which is how ACRS narrows the search. Categorical hyperparameters in the lower block are sampled from a fixed set of choices and are never contracted. The values are illustrative and do not stem from a concrete run.

Algorithm 2 details the PriorityOrder subroutine used in line 6 of Algorithm 1. The function produces a weighted random permutation of the n hyperparameter indices. Each index j is assigned a key $k_j = u_j / (w_j + \varepsilon)$, where $u_j \sim \text{Uniform}(0, 1)$ and ε prevents division by zero. Sorting the keys in ascending order yields the processing order π , where a high weight w_j produces a small expected key, so high-weight parameters tend to appear earlier in the ordering. The stochastic element ensures that no parameter is permanently locked out even if its weight is low.

Algorithm 2 Priority Order

Require: Number of hyperparameters n , weight vector $\mathbf{w} \in \mathbb{R}^n$, small constant $\varepsilon > 0$

- 1: **for** $j = 1$ **to** n **do**
 - 2: $u_j \sim \text{Uniform}(0, 1)$
 - 3: $k_j \leftarrow u_j / (w_j + \varepsilon)$
 - 4: **end for**
 - 5: **return** $\text{argsort}(k_1, \dots, k_n)$
-

The four variants evaluated in this work are summarized in Table 4.3. The two modifications follow straightforward intuitions. Range shrinking narrows the search interval around the current best value after each coordinate step, shifting the balance from exploration toward exploitation as optimization progresses. Gaussian sampling replaces the uniform draw with a normal distribution centered at the current best value λ_j^* , concentrating candidates near it while still allowing occasional excursions within the range. Both modifications are evaluated empirically to assess whether this increased exploitation bias improves performance under the given budget.

Tab. 4.3.: The table lists the four ACRS variants together with their flag settings.

Variant	<i>use_normal</i>	<i>shrink</i>
ACRS	false	false
ACRS (Normal)	true	false
ACRS (Shrink)	false	true
ACRS (Normal+Shrink)	true	true

4.2.2 Bayesian Optimization via TPE (BO)

BO uses a probabilistic surrogate model to guide the search toward promising regions of the hyperparameter space, concentrating evaluations where performance improvements are most likely rather than sampling blindly. The variant used in this work employs TPE [4] as its surrogate model, integrated via the Optuna framework [1]. The rationale for choosing TPE over GP-based BO is discussed in Section 2.3.

The TPE procedure maintains a growing observation set $\mathcal{D}$ of (configuration, performance) pairs and begins with a warm-up phase of random trials before fitting its density estimators. A quantile threshold γ partitions $\mathcal{D}$ into a good subset $\mathcal{D}_{\text{good}}$ (the top γ -fraction by performance) and a bad subset $\mathcal{D}_{\text{bad}}$, to which kernel density estimators $\ell(\lambda)$ and $g(\lambda)$ are fitted respectively. New candidates are selected by maximizing the ratio

$$\lambda^* = \arg \max_{\lambda \in \Lambda} \frac{\ell(\lambda)}{g(\lambda)}, \quad (4.2)$$

which Bergstra et al. [4] show to be equivalent to maximizing the Expected Improvement acquisition function, as a high ratio indicates that λ is more likely to come from the good observations than from the bad ones, and thus likely to improve upon the current best. Trials are executed sequentially to ensure that every new suggestion can condition on all previous observations. Optuna’s default TPE settings are used throughout; the full algorithm is given in Algorithm 3 in the Appendix. The budget T is set to match the ACRS evaluation budget; the concrete value is specified in Chapter 5.

4.2.3 Baseline Methods

Both methods are discussed in detail in Section 2.2. This section describes their formal mechanisms and their role in this benchmark. Like BO, both are implemented via the Optuna framework [1], while ACRS is implemented independently without Optuna.

RS [5] samples each configuration independently and uniformly at random from Λ , without using information from previous evaluations. Formally, at each trial $t \in \{1, \dots, T\}$ a configuration is drawn as

$$\lambda_t \sim \text{Uniform}(\Lambda), \quad (4.3)$$

and the best observed configuration $\lambda^* = \arg \max_t f(\lambda_t)$ is returned. The absence of any adaptive mechanism makes RS a natural lower bound for adaptive methods, as any improvement over it can be attributed directly to the use of past observations. It serves as a surrogate-free reference baseline that quantifies the gain attributable to the adaptive mechanisms of ACRS and BO.

CMA-ES [13, 12] extends this idea by learning from past evaluations. At each generation k , a population of N_{pop} candidate configurations is sampled from a multivariate Gaussian distribution,

$$\lambda_i^{(k)} \sim \mathcal{N}\left(\mu^{(k)}, (\sigma^{(k)})^2 C^{(k)}\right), \quad i = 1, \dots, N_{\text{pop}}, \quad (4.4)$$

where $\mu^{(k)} \in \mathbb{R}^n$ is the current mean, $\sigma^{(k)} > 0$ the global step size (distinct from the activation function σ used in Chapter 3), and $C^{(k)} \in \mathbb{R}^{n \times n}$ the covariance matrix. The top-ranked configurations by objective value are selected and used to update $\mu^{(k+1)}$ as a weighted average of the selected candidates. The covariance matrix $C^{(k+1)}$ is updated to align with the directions of recent successful steps, capturing dependencies between hyperparameters and progressively focusing sampling toward promising regions. CMA-ES thus serves as an adaptive surrogate-free baseline, enabling a comparison that isolates the contribution of ACRS's coordinatewise structure from what can be achieved by any adaptive sampling strategy.

4.3 Complexity Analysis

The cost of any HPO method is dominated by the number of objective function evaluations. To frame the scale, it is instructive to consider what exhaustive search would require. Grid Search enumerates every element of the discretized configuration space. As defined in Section 3.1, the configuration space is $\Lambda = \Lambda_1 \times \dots \times \Lambda_n$, so a grid with k_i candidate values along dimension i requires

$$|\Lambda_{\text{grid}}| = \prod_{i=1}^n k_i \quad (4.5)$$

evaluations in total. For a uniform grid with k values per dimension, this reduces to $\mathcal{O}(k^n)$, which grows exponentially with n . To make this concrete, a minimal hypothetical grid can be constructed for each search space by selecting five values per continuous or integer dimension, matching the candidate count L used by ACRS, and retaining the full categorical sets. For the MLP ($n = 8$), this yields at least

$$\underbrace{5}_{\text{lr}} \times \underbrace{4}_{\text{bs}} \times \underbrace{5}_{\text{hd}} \times \underbrace{5}_{\text{do}} \times \underbrace{3}_{\text{nl}} \times \underbrace{3}_{\text{opt}} \times \underbrace{5}_{\text{wd}} \times \underbrace{2}_{\text{act}} = 45,000 \text{ evaluations.} \quad (4.6)$$

For the RF ($n = 7$), the analogous minimal grid requires at least

$$\underbrace{5}_{\text{ne}} \times \underbrace{5}_{\text{md}} \times \underbrace{5}_{\text{mss}} \times \underbrace{2}_{\text{mf}} \times \underbrace{5}_{\text{msl}} \times \underbrace{2}_{\text{cr}} \times \underbrace{5}_{\text{ms}} = 12,500 \text{ evaluations.} \quad (4.7)$$

These figures are lower bounds, since any finer discretization of the continuous and integer dimensions increases the product proportionally.

By contrast, all four methods evaluated in this work operate with a fixed budget T that is at least two orders of magnitude smaller than these grid sizes. The budget for ACRS follows directly from Algorithm 1, as one evaluation is performed at initialization, and each of the R rounds draws L candidates per hyperparameter, giving

$$T_{\text{ACRS}} = 1 + R \cdot n \cdot L, \quad (4.8)$$

which is $\mathcal{O}(RnL)$ and thus scales linearly in each of its parameters, in direct contrast to the exponential $\mathcal{O}(k^n)$ of Grid Search. RS, CMA-ES, and BO are assigned the same budget T to ensure a fair comparison. The concrete values of R , L , and n (and thus the resulting T) are specified in Section 5.2.

Beyond the evaluation count, the methods differ in their per-trial overhead, that is, the computation performed between two evaluations to select the next candidate. RS requires only a single random draw per trial and carries no overhead. ACRS performs a weighted sort and a small number of scalar samples per round, which is negligible relative to training time. CMA-ES updates its $n \times n$ covariance matrix after each generation, at quadratic cost in n . BO refits its kernel density surrogate after every completed trial, with cost growing as observations accumulate. For the budgets used in this work, all of these overheads remain small compared to model training cost.

Benchmark Setup

The methods introduced in Chapter 4 are evaluated under a controlled setup designed to ensure that observed performance differences reflect the methods themselves rather than experimental artifacts. Section 5.1 introduces the five tabular datasets, selected to cover a broad range of problem characteristics. Section 5.2 specifies the evaluation budget, training protocols, and reproducibility measures.

5.1 Benchmark Datasets

Five tabular binary classification datasets are used in this thesis. They were selected to cover a broad range of characteristics, including sample sizes from 569 to nearly 49,000, feature dimensionalities from 8 to 105, and positive class rates from 11.7 % to 70.0 %, spanning from mild to severe class imbalance. This diversity ensures that results are not specific to a single dataset regime. Table 5.1 provides an overview. The positive class rate is reported because all five datasets exhibit at least some degree of class imbalance, which motivates the use of AUROC as the primary metric, as accuracy becomes an unreliable indicator of model quality when classes are unbalanced.

Tab. 5.1.: The table provides an overview of the five datasets used in the benchmark.

Dataset	Samples	Features	Positive (%)
Breast Cancer [25]	569	30	62.7
Diabetes [22]	768	8	34.9
German Credit [14]	1,000	61	70.0
Bank Marketing [19]	4,521*	51	11.7
Adult Income [2]	4,884*	105	23.9

* subsampled to 10% of the original dataset per run.

To keep the benchmark computationally tractable, a random 10 % subsample is drawn before splitting using the run seed, so the subset is fixed per seed but varies across seeds. Categorical features in Bank Marketing, German Credit, and Adult Income are converted to numerical form via one-hot encoding before training, since both the MLP and the RF require numerical inputs. The feature counts in Table 5.1 reflect the dimensionality after encoding. All datasets are then split into training, validation, and test sets in a stratified 60 / 20 / 20 ratio, preserving the class distribution across all three splits. The validation set is used exclusively as the HPO target. The test set is held out and

used only for final evaluation, so that repeated HPO trials cannot implicitly adapt to the test distribution and inflate the reported performance. Feature values are standardized with a `StandardScaler` fitted on the training set and applied to the validation and test sets, preventing any information leakage from later splits into the training process.

5.2 Benchmark Design

The benchmark compares seven optimization methods on both the MLP and the RF across all five datasets, namely BO, ACRS, ACRS (Normal), ACRS (Shrink), ACRS (Normal+Shrink), RS, and CMA-ES. The selection spans the full spectrum from surrogate-free to surrogate-based search. ACRS and its variants constitute the proposed method, BO via TPE serves as the main surrogate-based competitor, and RS and CMA-ES provide surrogate-free baselines.

The per-run evaluation budget is determined by the ACRS formula $T = 1 + R \cdot n \cdot L$, with $R = 3$ rounds, $L = 5$ candidate samples per coordinate step, and shrinking factor $\alpha = 0.5$, yielding $T = 121$ trials per MLP run ($n = 8$ hyperparameters) and $T = 106$ trials per RF run ($n = 7$). These values were chosen to give ACRS’s adaptive mechanisms enough iterations to operate while keeping T computationally tractable. With $R = 3$ rounds and $L = 5$ candidates, each hyperparameter receives 15 evaluations across the run, and $\alpha = 0.5$ halves the active search range on each shrinking step. All other methods use the same budget T to ensure a fair comparison (see Section 4.3). Since T follows directly from the ACRS algorithm structure, this provides a principled basis for the comparison, as all methods receive the same number of trials. The implications for methods that require a warm-up phase are discussed in Section 7.4. Each combination of method, dataset, and model is repeated with ten random seeds $\{42, 123, 456, 789, 1234, 2345, 3456, 4567, 5678, 6789\}$, yielding $7 \text{ methods} \times 5 \text{ datasets} \times 10 \text{ seeds} = 350$ runs per model type, 700 runs in total, and 79,450 total model evaluations. Ten seeds were chosen as a compromise between statistical reliability and computational feasibility. The resulting 700 runs already require 79,450 model evaluations, and increasing the seed count further was not feasible within the available compute budget. Table 5.2 summarizes the full experimental design.

Tab. 5.2.: The table summarizes the full benchmark design.

Model	Methods	Seeds	Datasets	Trials/Run	Runs	Evaluations
MLP	7	10	5	121	350	42,350
RF	7	10	5	106	350	37,100
Total					700	79,450

MLP models are trained for up to 100 epochs with early stopping applied when the validation AUROC does not improve for 10 consecutive epochs. Weights are initialized using PyTorch’s default Kaiming Uniform initialization.

The batch size is parameterized as 2^e with the exponent $e \in [4, 7]$ sampled uniformly, yielding values in $\{16, 32, 64, 128\}$, so the effective batch sizes are evenly spaced on a logarithmic scale (see Table 4.1).

The RF has no epoch-based training and is fitted in a single pass. The HPO target metric is validation AUROC, which is maximized by all optimization methods. After each HPO run, the best-found configuration is retrained and evaluated on the held-out test set, which is not used at any point during optimization. Results across seeds are aggregated as mean $\pm$ standard deviation.

To ensure full reproducibility, each run is seeded consistently across all sources of randomness, including the data split and subsampling, the PyTorch random number generator for MLP weight initialization and training, the RF's bootstrap sampling, and the internal random state of each optimization method. The ten seeds determine distinct data partitions and training conditions, so the variance across seeds reflects genuine algorithmic variability rather than measurement noise.

All experiments were conducted on an AMD Ryzen 5 5600 (6 cores) with 32 GB of RAM. The implementation uses Python 3.11, PyTorch 2.10.0 for MLP training, scikit-learn 1.8.0 for the RF, Optuna 4.7.0 for TPE, RS, and CMA-ES, NumPy 2.4.2 and pandas 3.0.0 for data loading and preprocessing, and CodeCarbon 3.2.2 for energy measurement. The full source code is available at <https://github.com/Dennis-Diehl/hyperparameter-optimization>.

Results

ACRS is designed as a surrogate-free alternative to BO (TPE), replacing the probabilistic surrogate model with coordinate-wise random search. Whether that design achieves comparable model quality at lower cost is the central question this chapter answers, evaluating final Test-AUROC, convergence speed, and total resource cost, where quality takes precedence but large efficiency gains can offset small quality differences. Section 6.1 first reduces ACRS from four variants to a single representative before the main comparison begins. Sections 6.2, 6.3, and 6.4 then compare this representative against BO (TPE), CMA-ES, and RS on final Test-AUROC, convergence behavior, and resource efficiency.

6.1 ACRS Ablation Study

ACRS comprises four variants defined by two orthogonal design flags (Table 4.3). Carrying all four into the main comparison would inflate the comparison tables with near-identical columns and divert attention from the central question of how ACRS performs against BO and the baselines. The four variants are therefore first compared internally on the full benchmark to select a single representative, which is then carried into all subsequent comparisons. Table 6.1 reports the pooled Test-AUROC of each variant.

Tab. 6.1.: Each cell reports the mean Test-AUROC and its standard deviation for one ACRS variant, pooled over the five datasets and ten seeds. The highest value in each row is highlighted in bold.

	ACRS	ACRS (Normal)	ACRS (Shrink)	ACRS (Normal+Shrink)
MLP	0.869 $\pm$ 0.084	0.869 $\pm$ 0.084	0.874 $\pm$ 0.081	0.868 $\pm$ 0.085
RF	0.881 $\pm$ 0.077	0.880 $\pm$ 0.079	0.882 $\pm$ 0.077	0.881 $\pm$ 0.078

ACRS (Shrink) attains the highest mean Test-AUROC for both models, reaching 0.874 for the MLP and 0.882 for the RF. The variants differ by at most 0.005, which is smaller than the run-to-run variation induced by the random seed of about 0.02,¹ so they are effectively indistinguishable in final model quality. The larger pooled standard deviation in Table 6.1 reflects differences in dataset difficulty rather than the variability between variants. ACRS (Shrink) is nonetheless selected as the representative variant, as it ranks first for both models. For the remainder of this work, ACRS refers to ACRS (Shrink).

¹Computed as the standard deviation of Test-AUROC across the ten seeds within each dataset, averaged over the five datasets.

6.2 Model Quality

The most fundamental question for a hyperparameter optimizer is how good the model it ultimately returns is. This section compares the four methods on the final quality of their selected configuration, measured by Test-AUROC. If they prove comparable here, the comparison is decided instead by how quickly (Section 6.3) and how cheaply (Section 6.4) they reach that quality. Table 6.2 pools Val-AUROC and Test-AUROC over all settings for the overall picture, the gap between the two showing how well the selected configurations generalize to unseen data, while Table 6.3 breaks the Test-AUROC down per dataset and model.

Tab. 6.2.: Each cell reports the mean Val-AUROC and Test-AUROC with their standard deviation per method and model, pooled over the five datasets and ten seeds. The best value in each row is highlighted in bold.

Modell	Metric	ACRS	BO (TPE)	CMA-ES	RS
MLP	Val-AUROC	0.901 ± 0.061	0.902 ± 0.060	0.900 ± 0.061	0.900 ± 0.061
MLP	Test-AUROC	0.874 ± 0.081	0.873 ± 0.083	0.871 ± 0.084	0.870 ± 0.083
RF	Val-AUROC	0.899 ± 0.062	0.901 ± 0.062	0.899 ± 0.062	0.899 ± 0.062
RF	Test-AUROC	0.882 ± 0.077	0.882 ± 0.077	0.881 ± 0.078	0.881 ± 0.077

Pooled over all datasets and seeds, the four methods are nearly identical (Table 6.2). On Val-AUROC, BO (TPE) is highest for both models, 0.902 for the MLP and 0.901 for the RF, with ACRS trailing by at most 0.002. On Test-AUROC the ordering inverts, as ACRS is highest for the MLP at 0.874 and ties BO (TPE) for the RF at 0.882. The largest gap between any two methods is 0.004, below the across-seed standard deviation of about 0.02. Test-AUROC falls below Val-AUROC for every method and both models, each losing a similar amount within its model, about 0.03 for the MLP and 0.02 for the RF.

Tab. 6.3.: Each cell reports the mean Test-AUROC and its standard deviation over the ten seeds per method, model, and dataset. The best value in each row is highlighted in bold.

Dataset	Modell	ACRS	BO (TPE)	CMA-ES	RS
Adult Income	MLP	0.887 ± 0.005	0.888 ± 0.007	0.890 ± 0.007	0.891 ± 0.006
Adult Income	RF	0.902 ± 0.008	0.901 ± 0.008	0.902 ± 0.008	0.902 ± 0.009
Bank Marketing	MLP	0.899 ± 0.014	0.896 ± 0.015	0.895 ± 0.006	0.895 ± 0.016
Bank Marketing	RF	0.913 ± 0.013	0.914 ± 0.014	0.913 ± 0.013	0.914 ± 0.013
Breast Cancer	MLP	0.995 ± 0.004	0.996 ± 0.004	0.994 ± 0.007	0.992 ± 0.008
Breast Cancer	RF	0.991 ± 0.005	0.991 ± 0.005	0.991 ± 0.005	0.990 ± 0.006
Diabetes	MLP	0.825 ± 0.027	0.824 ± 0.029	0.822 ± 0.025	0.814 ± 0.032
Diabetes	RF	0.827 ± 0.022	0.826 ± 0.019	0.828 ± 0.023	0.824 ± 0.018
German Credit	MLP	0.764 ± 0.037	0.759 ± 0.035	0.754 ± 0.037	0.757 ± 0.022
German Credit	RF	0.775 ± 0.027	0.775 ± 0.024	0.773 ± 0.030	0.776 ± 0.029

Resolving the Test-AUROC per dataset and model shows whether the pooled near-tie holds in each individual setting. It does, with the best value changing from row to row and no method dominating. ACRS attains or shares the top score in five of the ten dataset-model combinations, CMA-ES and RS in three each, and BO (TPE) in two. Within a row the methods differ by at most 0.011, reached for the MLP on Diabetes, where the across-seed standard deviation is itself about 0.03. This spread stays below it in every row. Across datasets the spread is far wider, about 0.24 for the MLP and 0.22 for the RF, lowest on German Credit and highest on Breast Cancer in both models.

6.3 Convergence Behavior

Since the four methods reach comparable final quality (Section 6.2), the comparison turns to how quickly each reaches it. Convergence is measured as the best Val-AUROC found so far, tracked against both the trial number (Figure 6.1) and the cumulative training time (Figure 6.2). These two views can diverge. Strong convergence per trial can be misleading when each trial is slow, since a method that needs more trials but runs them cheaply may still finish sooner. The per-trial training time underlying the time view is what later accumulates into the overall cost of a search. Both figures show Adult Income and German Credit, chosen to cover a difficult and a costly case. German Credit is the hardest dataset, with the lowest AUROC (Table 6.3), while Adult Income scores high but is the most expensive to train (Table A.1). Breast Cancer, the easiest, saturates almost immediately and would reveal little. The remaining datasets follow the same trend, with all five shown with ± 1 across-seed standard deviation bands in Figures A.2 and A.3.

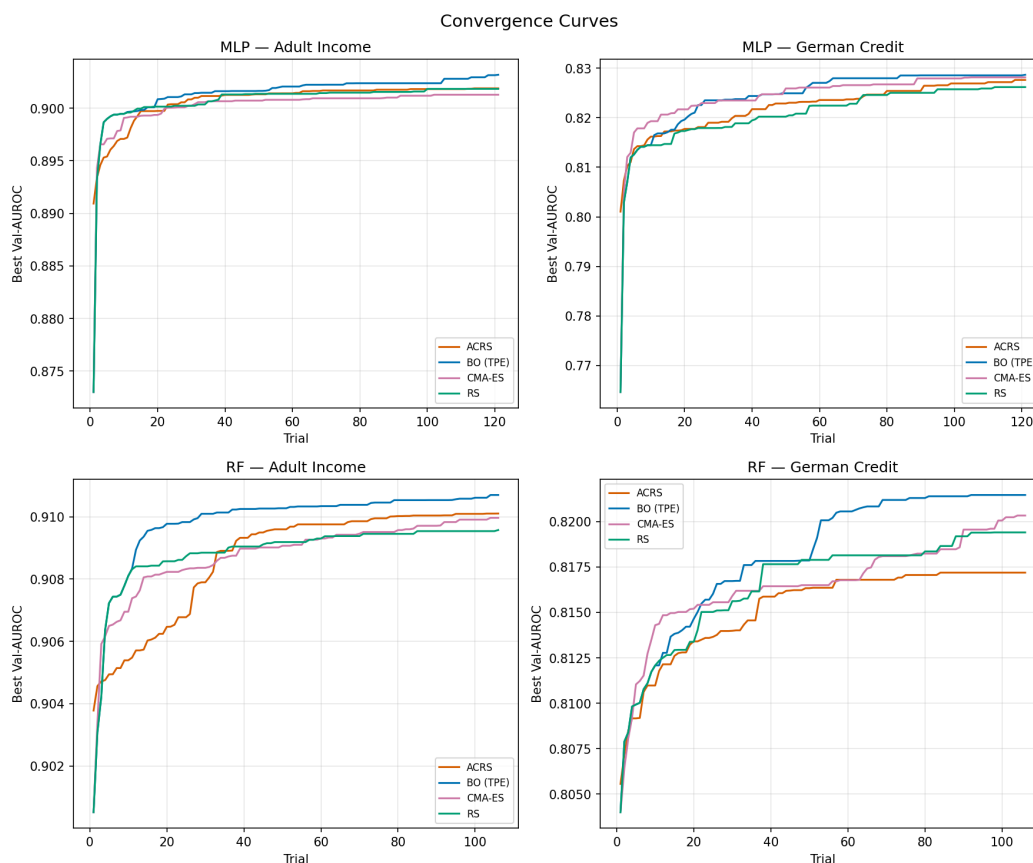


Fig. 6.1.: The best Val-AUROC achieved so far is plotted against the trial number, averaged over the ten seeds, with one row per model (MLP top, RF bottom). Only the mean curves are shown, for the two representative datasets Adult Income and German Credit.

Per trial, all four methods rise steeply and converge into a narrow band, though the fine structure differs by model. For the MLP the band closes

especially tightly, the methods ending within about 0.003 on both datasets, after levelling off around trial 30 on Adult Income and more slowly on German Credit. For the RF, BO (TPE) ends highest on both datasets, consistent with its marginally higher Val-AUROC in Section 6.2. ACRS rises almost linearly on Adult Income, starting below the others and catching up to just under BO (TPE), while on German Credit it settles lowest, about 0.004 below BO (TPE). These final gaps stay well below the across-seed standard deviation.

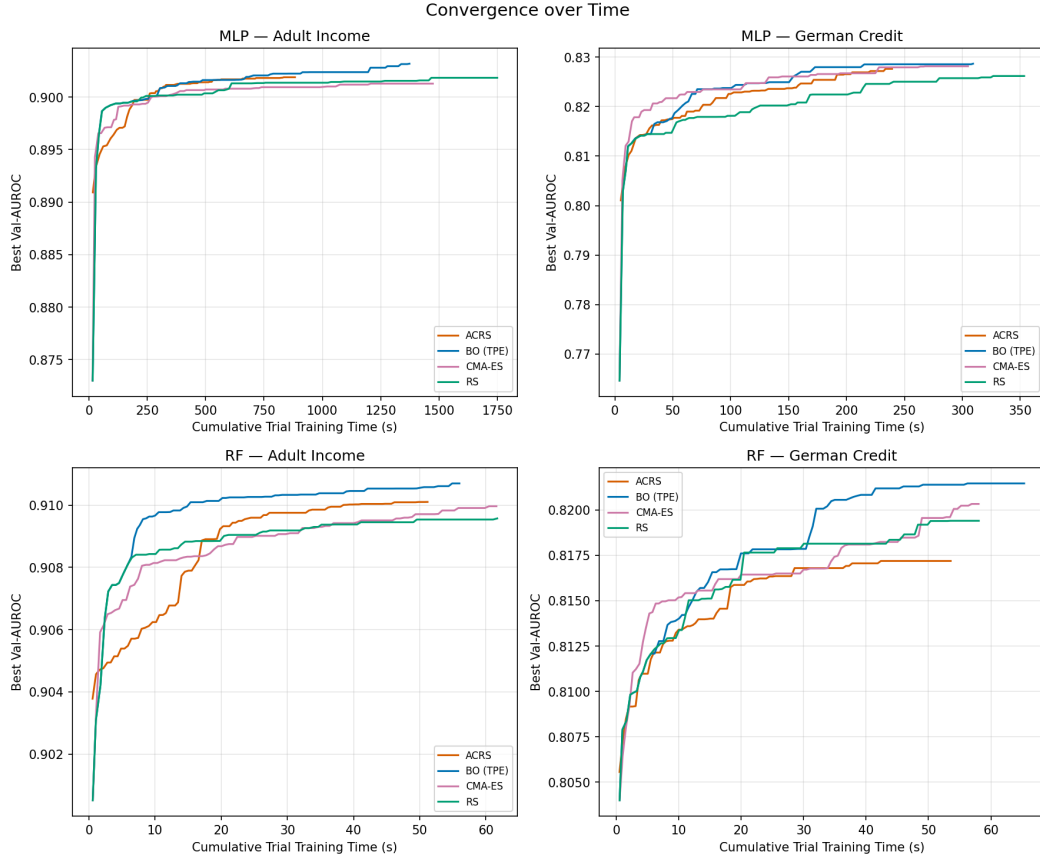


Fig. 6.2.: The best Val-AUROC achieved so far is plotted against the cumulative training time, averaged over the ten seeds, with one row per model (MLP top, RF bottom). Only the mean curves are shown, for the two representative datasets Adult Income and German Credit.

Against training time, the methods stay as close as before, with ACRS at times slightly below. The per-trial picture reverses, as ACRS, which never converged faster per trial, now terminates first in every case. The difference is largest for the MLP, where on Adult Income ACRS finishes near 900 seconds and the slowest, RS, near 1750 seconds, whereas for the much cheaper RF all methods finish within about a minute. ACRS thus reaches comparable Val-AUROC in the least training time, an advantage quantified as total cost in Section 6.4.

6.4 Resource Efficiency

Section 6.3 showed that ACRS terminates first against cumulative training time, despite not converging faster per trial. Total run duration and energy consumption extend that picture by including method-internal overhead beyond individual training runs. BO (TPE) and CMA-ES fit a probabilistic surrogate model between each trial, and that fitting cost accumulates over the full search. Table 6.4 reports both metrics, pooled over all settings, to measure how much surrogate-fitting overhead widens the cost gap observed in Section 6.3. Table A.1 in the Appendix provides the per-dataset and per-model breakdown.

Tab. 6.4.: Each cell reports the resource efficiency per method and model, pooled over the five datasets and ten seeds, given as the mean and standard deviation. The two metrics are the total run duration in seconds and the total energy consumption in kWh, the latter measured with CodeCarbon [8]. The best (lowest) value in each row is highlighted in bold.

Modell	Metric	ACRS	BO (TPE)	CMA-ES	RS
MLP	Duration (s)	477.5 ± 497.2	655.9 ± 630.9	636.2 ± 604.0	799.4 ± 686.2
MLP	Energy (kWh)	0.0541 ± 0.0563	0.0743 ± 0.0715	0.0721 ± 0.0684	0.0906 ± 0.0777
RF	Duration (s)	51.2 ± 21.4	57.7 ± 14.3	58.0 ± 11.2	57.8 ± 4.0
RF	Energy (kWh)	0.0058 ± 0.0024	0.0065 ± 0.0016	0.0066 ± 0.0013	0.0066 ± 0.0005

ACRS leads in both duration and energy for the MLP and the RF. For the MLP the gap is substantial, as CMA-ES, BO (TPE), and RS require 33–67 % more time, with energy consumption mirroring the duration ordering exactly. For the RF the advantage narrows to roughly 13 % across all three competing methods, with energy again following the same ordering. The large standard deviations for the MLP, where the pooled spread can exceed the mean, reflect dataset heterogeneity rather than method instability, as Table A.1 confirms. Breast Cancer evaluates in seconds while Adult Income can take several minutes per trial. Notably, RS is the most expensive method for the MLP despite incurring no surrogate-fitting overhead.

Discussion

This thesis proposes ACRS as a surrogate-free alternative to BO and asks whether it achieves comparable model quality and whether it does so at lower resource cost. The results in Chapter 6 provide concrete answers to both. This chapter interprets those answers. Section 7.1 examines what the near-uniform quality across methods means, Section 7.2 explains why ACRS finishes faster despite converging at the same rate per trial, Section 7.3 interprets the marginal effect of the ACRS design choices, and Section 7.4 identifies the boundaries within which these conclusions hold.

7.1 Interpretation of Model Quality

Quality parity across all four methods arises because the search spaces are dominated by a small number of hyperparameters, a condition under which any method with a sufficient budget converges to near-optimal configurations. Bergstra and Bengio [5] show that random search already saturates performance in this regime, as confirmed by Table 6.2. No method deviates from the best-performing one by more than the across-seed standard deviation, making all differences statistically indistinguishable in practice. The quality parity nonetheless supports the efficiency-based argument in Section 7.2, since a clear quality advantage of one method over another would complicate a cost comparison.

This outcome matches the observation by Yang and Shami [28] that the relative ranking of HPO methods varies across tasks and budgets, with no single method consistently dominating. The result further aligns with ACRS’s design rationale, as Nutini et al. [20] prove that priority-weighted coordinate selection converges no slower than random coordinate selection, and Florea and Andonie [11] show that importance-weighted random search has a probability of reaching the global optimum at least as high as standard RS. Both predictions are consistent with ACRS matching or exceeding RS across all dataset-model combinations and matching BO on Test-AUROC (Table 6.3). ACRS is designed to benefit from its weight-based prioritization by concentrating evaluations on the dimensions that produced the largest gains in earlier rounds. Indirect support for this mechanism comes from the efficiency comparison in Table 6.4, where ACRS consistently finishes faster than RS for the MLP, which is consistent with the coordinatewise and weighting mechanisms together steering evaluations toward lower-cost regions of the search space. That this does not translate into a measurably larger quality gap reflects the budget constraint discussed in Section 7.4, where the small number of rounds limits how far the adaptive weights can compound.

BO optimizes its TPE surrogate directly against Val-AUROC, so it tends to select configurations that score well there. This focus may explain why BO leads on Val-AUROC for both models while the ordering on Test-AUROC inverts in favour of ACRS, which does not fit a surrogate to the validation signal. The absolute differences are well within the across-seed standard deviation and no causal interpretation is warranted at this margin, but the pattern is consistent across both model types and noted here for completeness.

7.2 Efficiency Advantage and Its Mechanism

The efficiency advantage of ACRS stems from its coordinatewise structure. By fixing all hyperparameters except the one currently being optimized at their current best values, each trial evaluates a configuration that is close to the known best rather than a draw from the full search space. For the MLP, individual trial cost is highly sensitive to hidden dimension and batch size, since a wide network trained with small batches requires many more floating-point operations per epoch than a narrow network with large batches. Coordinatewise search therefore tends to avoid the expensive end of the configuration space once a reasonable baseline is established, keeping average trial cost low throughout the run. This cost reduction is what drives the substantial differences visible in Table 6.4.

The advantage does not stem from faster per-trial convergence, as Figure 6.1 shows all four methods rising into the same narrow band at roughly the same rate when plotted against trial number. Against cumulative training time, however, ACRS terminates first in every case (Figure 6.2). The large standard deviations in Table 6.4 reflect dataset heterogeneity rather than run-to-run instability. Breast Cancer completes in seconds while Adult Income can take several minutes per trial, as the per-dataset breakdown in Table A.1 confirms.

The most counterintuitive consequence of this mechanism is that RS ends up as the most expensive method for the MLP despite incurring no surrogate-fitting overhead. RS draws each trial independently and uniformly from the full search space, continuously sampling configurations across the entire cost spectrum throughout the run, consistent with its design as a memoryless baseline [5]. The absence of overhead does not compensate for the cost of continuously drawing expensive configurations that guided methods increasingly avoid as the search progresses. CMA-ES, which refits its full $n \times n$ covariance matrix after each generation, incurs intermediate overhead relative to BO and RS. Its position between BO and RS in Table 6.4 is consistent with this structure, as the covariance update is cheaper than TPE's sequential kernel density refitting but adds non-trivial cost that RS avoids entirely [12].

The adaptive weight mechanism underlying the ACRS efficiency finds empirical support in the work of Hutter et al. [15], who show that performance variation is typically dominated by a few hyperparameters even in high-dimensional spaces. Once ACRS identifies these through its weight updates

(Algorithm 1, lines 23–27), it concentrates most evaluations on the influential dimensions and processes the remaining hyperparameters with the current best values fixed, keeping those trials computationally inexpensive. All four methods show steepest gains early and level off as the budget is consumed, a pattern common to HPO under a fixed budget [5], but ACRS achieves the same plateau faster in cumulative training time precisely because later trials, guided by accumulated weights, tend to involve less expensive configurations.

For the RF, training cost is relatively uniform across configurations, so varying `n_estimators` or `max_depth` changes training time only marginally compared to the MLP. The cost heterogeneity that drives the MLP efficiency advantage is therefore largely absent, and the gap between methods narrows accordingly (Table 6.4). This indicates that the cost structure of the model being tuned governs the size of the ACRS efficiency advantage rather than its presence, as ACRS remains the fastest method for both models in Table 6.4.

Taken together, these observations suggest the conditions under which the ACRS efficiency advantage is largest, namely when individual trial costs are heterogeneous and surrogate-fitting overhead is non-negligible relative to training time. For practitioners running sequential HPO on models with heterogeneous training costs and without surrogate infrastructure, the results of this thesis suggest ACRS as a resource-efficient alternative that does not sacrifice model quality.

7.3 Ablation: Effect of the ACRS Design Choices

The ablation reveals that the two experimental extensions of ACRS, namely range shrinking and Gaussian candidate sampling, do not produce measurably different outcomes under the experimental conditions of this thesis. Two explanations are consistent with this finding, and the available data do not distinguish between them. First, $R = 3$ rounds may be insufficient for the adaptive mechanisms to differentiate, as shrinking, Gaussian sampling, and importance weighting all require multiple iterations to compound, and the differences between strategies cannot manifest within three passes over the hyperparameters. Second, the tabular search spaces used here may contain few strong hyperparameter interactions, in which case coordinatewise optimization already performs near-optimally regardless of variant, consistent with the finding by Hutter et al. [15] that performance is often dominated by a small number of dimensions that any coordinatewise strategy will identify quickly. Determining which explanation dominates would require either a larger- R experiment or a benchmark with a known hyperparameter interaction structure.

The marginal effect of range shrinking is further interpretable in the context of the experimental budget. Range shrinking is most beneficial when multiple rounds allow progressive refinement, with each round exploiting the narrowed range established by the previous one. At $R = 3$, this compounding

effect has little room to develop, and the shrinking factor $\alpha = 0.5$ means the range is already halved after a single application. A larger number of rounds or a larger candidate count L would likely produce a more pronounced exploitation benefit, in line with the first explanation above that $R = 3$ rounds may be too few for the adaptive mechanisms to differentiate.

The absence of an effect from Gaussian sampling follows a similar logic, as concentrating the $L = 5$ candidates per coordinate step near the current best reduces coverage of the range at the cost of finer local resolution, a trade-off that does not pay off at this sample count. Importance weighting is not an ablated variant but a fixed component present in all ACRS configurations, grounded in the design rationale of Florea and Andonie [11], so its contribution is not separately quantified here. The same $R = 3$ constraint applies nonetheless, as the weight vector is updated only twice and the first round always operates on uniform weights, limiting how strongly the prioritization can differentiate between hyperparameters within the available rounds.

Overall, these results indicate that under the budget used in this thesis, none of the additional ACRS mechanisms, namely shrinking, Gaussian sampling, and the differentiation of importance weights, are given enough rounds or candidates to produce a measurable advantage over plain coordinatewise search. ACRS (Shrink) is nonetheless selected as the representative variant for the main comparison, as it ranks first for both models in Table 6.1 and its shrinking-based interpretation aligns with the design intent of progressively narrowing the search.

7.4 Limitations

The evaluation budget follows the ACRS formula $T = 1 + R \cdot n \cdot L$, yielding $T = 121$ for the MLP and $T = 106$ for the RF, and is shared across all methods (Section 5.2). It does, however, favor ACRS, as surrogate-based methods such as BO require a warm-up phase of random initial trials before their model of the objective function becomes informative enough to guide candidate selection. At $T = 121$, this warm-up consumes a substantial share of the budget, which means BO operates closer to random search than to its full surrogate-guided mode for much of the run [10]. The observed quality parity and efficiency advantage of ACRS therefore hold under this specific budget regime and do not imply a general ranking, since at larger budgets BO's surrogate would compound across more observations and the comparison might shift.

The same $R = 3$ constraint discussed in Section 7.3 limits how far ACRS's adaptive weighting and range shrinking can develop within a single run, leaving open whether a larger R would yield measurably better final quality.

CMA-ES faces a similar budget constraint. Its default population size grows logarithmically with the search space dimension and amounts to nine to ten candidates per generation here [12], so the shared budget covers only about twelve generations. The covariance matrix therefore receives few updates

before the budget is exhausted, leaving little room for exploitation. Beyond the budget, CMA-ES is designed for continuous, unconstrained optimization and has no principled representation for categorical hyperparameters [12]. The search spaces were deliberately designed to reduce this mismatch, for instance by encoding batch size as a continuous exponent `batch_size_exp` rather than a categorical set, but several categorical hyperparameters remain (Tables 4.1 and 4.2) that the covariance matrix cannot meaningfully model and that Optuna’s CMA-ES implementation instead selects through independent random sampling. Both constraints together mean that the benchmark setting is structurally more favourable to ACRS, which handles mixed search spaces without modification, than to CMA-ES. The observed quality parity therefore provides conservative evidence for ACRS’s competitiveness rather than a tight upper bound on CMA-ES’s potential.

With 10 seeds per configuration, the across-seed standard deviation in Test-AUROC is approximately 0.02, and all observed pairwise differences between methods fall below this threshold. No formal pairwise significance test was conducted. Given this across-seed standard deviation, such a test is unlikely to reach significance under standard thresholds, and the quality comparison should be interpreted as evidence of practical equivalence rather than proven equality. Increasing the number of seeds would reduce this uncertainty, but was not feasible within the available computational budget.

The benchmark covers five tabular binary classification datasets, two of which are subsampled to 10 % of their original size for computational tractability. This subsampling reduces the size and difficulty of the affected tasks, which may lower the absolute performance levels reported for those datasets. It does not, however, explain differences in method ranking, which remains consistent with the three datasets used at full size (Table 6.3). The conclusions do not generalize to regression tasks, multi-class problems, image or text data, or large-scale deep learning benchmarks where the cost structure and search space characteristics differ substantially. The HPO literature has noted that results on small tabular datasets do not transfer reliably to larger-scale settings [10, 28], and this benchmark should be understood accordingly.

Timing and energy measurements were obtained on a single hardware configuration (AMD Ryzen 5 5600). On such homogeneous CPU-only hardware, energy consumption is approximately proportional to elapsed time and mirrors the duration ordering (Table 6.4), so the energy metric adds little independent information here and becomes informative in its own right only on heterogeneous hardware with diverging power profiles. The relative ordering of methods by total run duration may also differ on GPU hardware, where training cost is lower and surrogate-fitting overhead constitutes a larger fraction of total time, or in parallelized settings, where RS is trivially parallelizable while ACRS’s sequential coordinate structure is not.

In summary, the conclusions of Sections 7.1 and 7.2 hold within the budget, sample-size, dataset, and hardware scope outlined above. Extending them to substantially larger budgets, additional seeds, other data domains, or GPU and parallelized hardware would require separate validation.

Conclusion

This thesis evaluated ACRS, a surrogate-free optimizer that combines coordinatewise search with adaptive importance weighting, against BO (TPE) for tuning MLP and RF classifiers, asking two questions: whether ACRS matches or exceeds BO’s model quality, and whether it does so at lower computational cost. Section 8.1 summarizes the findings of Chapters 6 and 7 and answers both questions, while Section 8.2 discusses the practical implications and directions for future work.

8.1 Summary of Findings

ACRS optimizes one hyperparameter at a time while holding the others fixed, following the coordinate descent principle [27], and maintains adaptive weights that prioritize hyperparameters by their observed contribution to performance, a strategy supported by Nutini et al. [20] and Florea and Andonie [11] and motivated by the finding of Hutter et al. [15] that performance variation in HPO is typically dominated by a few hyperparameters. The resulting budget $T = 1 + R \cdot n \cdot L$ scales linearly in n , unlike the $\mathcal{O}(k^n)$ growth of Grid Search, and was applied to all four methods across $n = 8$ (MLP) and $n = 7$ (RF) hyperparameters, five datasets, and ten seeds.

The ablation (Section 6.1) found that ACRS (Shrink) achieves the highest mean Test-AUROC for both models, 0.874 for the MLP and 0.882 for the RF, and was selected as the representative variant. Pooled over all settings, the four methods proved nearly indistinguishable in final quality, ACRS matches or exceeds BO (TPE) on Test-AUROC for both models, and all pairwise differences fall within the across-seed standard deviation of about 0.02, consistent with the No Free Lunch theorem [26] and with Bergstra and Bengio’s [5] finding that RS already saturates performance in search spaces dominated by few hyperparameters.

The central finding concerns resource efficiency. ACRS leads in total run duration and energy consumption for both models (Section 6.4), with the other methods requiring 33–67 % more time for the MLP and about 13 % more for the RF. This advantage does not stem from faster per-trial convergence, since all four methods reach the same quality plateau at a similar rate per trial, but instead from ACRS’s coordinatewise structure systematically avoiding the most expensive configurations once a reasonable baseline is established (Section 7.2). Its size depends on the cost structure of the model, largest where trial costs vary widely as for the MLP and smaller where they are uniform as for the RF, though ACRS remained the fastest method for both.

These results answer the question posed at the outset of this thesis. Within the budget regime studied here, ACRS matches the model quality of BO (TPE) and CMA-ES for both models while consuming less time and energy, without fitting any surrogate model. This scope cuts both ways: at larger budgets, BO's surrogate could compound across more observations and pull ahead on quality, while ACRS's own adaptive mechanisms, themselves limited to $R = 3$ rounds, might likewise yield a larger advantage with more rounds. The efficiency advantage is real but model-dependent, larger when training costs vary across configurations and smaller when they do not, while the quality match holds independently of this distinction.

8.2 Practical Implications and Future Work

For practitioners, this suggests a simple decision rule. When tuning a model whose training cost varies substantially across configurations, such as an MLP where hidden dimension and batch size strongly affect per-epoch cost, and no infrastructure for fitting a surrogate model is already in place, ACRS offers a lightweight alternative to BO that reaches comparable Test-AUROC at meaningfully lower computational and energy cost. When training cost is comparatively uniform, as for the RF, the gain is smaller but still favors ACRS.

Several directions follow from the limitations discussed in Section 7.4. The ablation could not determine whether range shrinking, Gaussian sampling, and the differentiation of importance weights remain ineffective under a larger budget or whether these search spaces simply lack strong hyperparameter interactions, a question that a larger R or L could resolve. More seeds would tighten the across-seed standard deviation and might allow the remaining Test-AUROC differences, at most 0.011, to be assessed with a formal significance test. Repeating the comparison at larger budgets would clarify whether it shifts in BO's favor once its warm-up phase represents a smaller share of T , and extending it to regression, multi-class, or GPU-trained models would test how far these conclusions generalize. Finally, since ACRS's sequential coordinatewise structure, unlike RS, is not trivially parallelizable, a variant that evaluates its L candidates per coordinate step concurrently could combine importance-weighted prioritization with better support for parallel hardware.

Bibliography

- [1]Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019, pp. 2623–2631 (cit. on pp. 5, 19, 20).
- [2]Barry Becker and Ronny Kohavi. *Adult*. UCI Machine Learning Repository. 1996 (cit. on p. 23).
- [3]Yoshua Bengio, Ian Goodfellow, Aaron Courville, et al. *Deep learning*. Vol. 1. MIT press Cambridge, MA, USA, 2016 (cit. on pp. 9, 10).
- [4]James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. “Algorithms for Hyper-Parameter Optimization”. In: *Advances in Neural Information Processing Systems 24 (NeurIPS)*. 2011, pp. 2546–2554 (cit. on pp. 4, 19).
- [5]James Bergstra and Yoshua Bengio. “Random search for hyper-parameter optimization.” In: *Journal of machine learning research* 13.2 (2012), pp. 281–305 (cit. on pp. 4, 5, 20, 33–35, 39).
- [6]Leo Breiman. “Bagging Predictors”. In: *Machine Learning* 24.2 (1996), pp. 123–140 (cit. on p. 11).
- [7]Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32 (cit. on p. 10).
- [8]Benoit Courty, Victor Schmidt, Goyal-Kamal, et al. *mlco2/codecarbon: v3.2.6*. Version v3.2.6. 2026 (cit. on pp. 1, 2, 6, 9, 32).
- [9]Tom Fawcett. “An Introduction to ROC Analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874 (cit. on p. 8).
- [10]Matthias Feurer and Frank Hutter. “Hyperparameter Optimization”. In: *Automated Machine Learning: Methods, Systems, Challenges*. Ed. by Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren. Springer, 2019, pp. 3–33 (cit. on pp. 3, 4, 6, 7, 36, 37).
- [11]Adrian-Catalin Florea and Razvan Andonie. “Weighted random search for hyperparameter optimization”. In: *arXiv preprint arXiv:2004.01628* (2020) (cit. on pp. 1, 5, 15, 33, 36, 39).
- [12]Nikolaus Hansen. “The CMA Evolution Strategy: A Tutorial”. In: *arXiv preprint arXiv:1604.00772* (2016) (cit. on pp. 4, 20, 34, 36, 37).
- [13]Nikolaus Hansen and Andreas Ostermeier. “Completely Derandomized Self-Adaptation in Evolution Strategies”. In: *Evolutionary Computation* 9.2 (2001), pp. 159–195 (cit. on pp. 4, 20).
- [14]Hans Hofmann. *Statlog (German Credit Data)*. UCI Machine Learning Repository. 1994 (cit. on p. 23).
- [15]Frank Hutter, Holger Hoos, and Kevin Leyton-Brown. “An efficient approach for assessing hyperparameter importance”. In: *International conference on machine learning*. PMLR. 2014, pp. 754–762 (cit. on pp. 1, 5, 15, 34, 35, 39).

- [16]Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. “Sequential Model-Based Optimization for General Algorithm Configuration”. In: *Proceedings of the 5th International Conference on Learning and Intelligent Optimization (LION)*. 2011, pp. 507–523 (cit. on p. 4).
- [17]Diederik P. Kingma and Jimmy Ba. “Adam: A Method for Stochastic Optimization”. In: *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. 2015 (cit. on p. 10).
- [18]Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. “Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization”. In: *Journal of Machine Learning Research* 18 (2018), pp. 1–52 (cit. on pp. 3, 6).
- [19]Sérgio Moro, Paulo Rita, and Paulo Cortez. *Bank Marketing*. UCI Machine Learning Repository. 2014 (cit. on p. 23).
- [20]Julie Nutini, Mark Schmidt, Issam Laradji, Michael Friedlander, and Hoyt Koepke. “Coordinate descent converges faster with the gauss-southwell rule than random selection”. In: *International Conference on Machine Learning*. PMLR. 2015, pp. 1632–1641 (cit. on pp. 1, 5, 15, 33, 39).
- [21]Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. “Taking the Human Out of the Loop: A Review of Bayesian Optimization”. In: *Proceedings of the IEEE* 104.1 (2016), pp. 148–175 (cit. on pp. 3, 4).
- [22]J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes. “Using the ADAP Learning Algorithm to Forecast the Onset of Diabetes Mellitus”. In: *Proceedings of the Annual Symposium on Computer Applications in Medical Care*. 1988, pp. 261–265 (cit. on p. 23).
- [23]Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. “Practical Bayesian Optimization of Machine Learning Algorithms”. In: *Advances in Neural Information Processing Systems 25 (NeurIPS)*. 2012, pp. 2951–2959 (cit. on p. 4).
- [24]Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958 (cit. on p. 10).
- [25]William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. *Breast Cancer Wisconsin (Diagnostic)*. UCI Machine Learning Repository. 1993 (cit. on p. 23).
- [26]David H. Wolpert and William G. Macready. “No Free Lunch Theorems for Optimization”. In: *IEEE Transactions on Evolutionary Computation* 1.1 (1997), pp. 67–82 (cit. on pp. 6, 8, 39).
- [27]Stephen J. Wright. “Coordinate Descent Algorithms”. In: *Mathematical Programming* 151.1–2 (2015), pp. 3–34 (cit. on pp. 5, 39).
- [28]Li Yang and Abdallah Shami. “On Hyperparameter Optimization of Machine Learning Algorithms: Theory and Practice”. In: *Neurocomputing* 415 (2020), pp. 295–316 (cit. on pp. 3, 4, 6, 33, 37).

List of Figures

4.1.	Schematic illustration of the ACRS mechanism on the MLP search space. Each row is a hyperparameter and each column a round. For a continuous hyperparameter, the grey bar spans its full domain, the highlighted segment marks the current search range, and the dot marks the value selected in that round. From left to right the highlighted range contracts around the selected value, which is how ACRS narrows the search. Categorical hyperparameters in the lower block are sampled from a fixed set of choices and are never contracted. The values are illustrative and do not stem from a concrete run.	18
6.1.	The best Val-AUROC achieved so far is plotted against the trial number, averaged over the ten seeds, with one row per model (MLP top, RF bottom). Only the mean curves are shown, for the two representative datasets Adult Income and German Credit.	30
6.2.	The best Val-AUROC achieved so far is plotted against the cumulative training time, averaged over the ten seeds, with one row per model (MLP top, RF bottom). Only the mean curves are shown, for the two representative datasets Adult Income and German Credit.	31
A.1.	The empirical cumulative distribution function (ECDF) of the Test-AUROC is shown in two panels, one for the MLP and one for the RF.	48
A.2.	The best Val-AUROC achieved so far is plotted against the trial number, averaged over the ten seeds, with the shaded area denoting ± 1 across-seed standard deviation. Each row corresponds to one model (MLP top, RF bottom) and each column to one dataset. . . .	48
A.3.	The best Val-AUROC achieved so far is plotted against the cumulative training time, averaged over the ten seeds, with the shaded area denoting ± 1 across-seed standard deviation. Each row corresponds to one model (MLP top, RF bottom) and each column to one dataset.	49

List of Tables

4.1.	This table specifies the hyperparameter search space of the MLP. . .	13
4.2.	This table specifies the hyperparameter search space of the RF. . .	14
4.3.	The table lists the four ACRS variants together with their flag settings.	19
5.1.	The table provides an overview of the five datasets used in the benchmark.	23
5.2.	The table summarizes the full benchmark design.	24
6.1.	Each cell reports the mean Test-AUROC and its standard deviation for one ACRS variant, pooled over the five datasets and ten seeds. The highest value in each row is highlighted in bold.	27
6.2.	Each cell reports the mean Val-AUROC and Test-AUROC with their standard deviation per method and model, pooled over the five datasets and ten seeds. The best value in each row is highlighted in bold.	28
6.3.	Each cell reports the mean Test-AUROC and its standard deviation over the ten seeds per method, model, and dataset. The best value in each row is highlighted in bold.	28
6.4.	Each cell reports the resource efficiency per method and model, pooled over the five datasets and ten seeds, given as the mean and standard deviation. The two metrics are the total run duration in seconds and the total energy consumption in kWh, the latter measured with CodeCarbon [8]. The best (lowest) value in each row is highlighted in bold.	32
A.1.	The table reports the total run duration in seconds per method, model, and dataset, given as the mean and standard deviation over the ten seeds.	49

Appendix

A.1 Bayesian Optimization Algorithm

Algorithm 3 gives the complete pseudocode for the TPE-based BO procedure described in Section 4.2.2.

Algorithm 3 Bayesian Optimization via TPE

Require: Search space Λ , objective f , budget T , warm-up count n_{warmup} , quantile γ

```

1:  $\mathcal{D} \leftarrow \emptyset$ 
2: for  $t = 1$  to  $T$  do
3:   if  $t \leq n_{\text{warmup}}$  then
4:      $\lambda_t \leftarrow \text{Uniform}(\Lambda)$ 
5:   else
6:      $\mathcal{D}_{\text{good}} \leftarrow \text{top } \lceil \gamma \cdot |\mathcal{D}| \rceil \text{ observations in } \mathcal{D}$ 
7:      $\mathcal{D}_{\text{bad}} \leftarrow \mathcal{D} \setminus \mathcal{D}_{\text{good}}$ 
8:     Fit  $\ell(\lambda)$  on  $\mathcal{D}_{\text{good}}$ ; fit  $g(\lambda)$  on  $\mathcal{D}_{\text{bad}}$ 
9:      $\lambda_t \leftarrow \arg \max_{\lambda \in \Lambda} \ell(\lambda)/g(\lambda)$ 
10:  end if
11:   $y_t \leftarrow f(\lambda_t)$ 
12:   $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\lambda_t, y_t)\}$ 
13: end for
14: return  $\lambda^* = \arg \max_{(\lambda, y) \in \mathcal{D}} y$ 

```

Line 1 initializes an empty observation set $\mathcal{D}$; the main loop (lines 2–13) iterates over the budget T . During warm-up (lines 3–4), configurations are drawn uniformly from Λ to collect an initial set of observations before any density model can be fitted. After warm-up (lines 5–10), $\mathcal{D}$ is partitioned at quantile γ , where the top $\lceil \gamma \cdot |\mathcal{D}| \rceil$ observations by objective value form $\mathcal{D}_{\text{good}}$ and the remainder forms $\mathcal{D}_{\text{bad}}$. Kernel density estimators $\ell(\lambda)$ and $g(\lambda)$ are fitted to the two subsets, and the next candidate is selected as the configuration that maximizes the ratio $\ell(\lambda)/g(\lambda)$, preferring regions that are dense under the good observations and sparse under the bad ones. The selected candidate is evaluated on the objective (line 11) and the resulting pair (λ_t, y_t) is appended to $\mathcal{D}$ (line 12). After T trials, the algorithm returns the configuration with the highest observed performance (line 14).

A.2 Performance Distribution

Figure A.1 shows the Empirical Cumulative Distribution Function (ECDF) of Test-AUROC across all runs, complementing the pooled performance summary in Table 6.2.

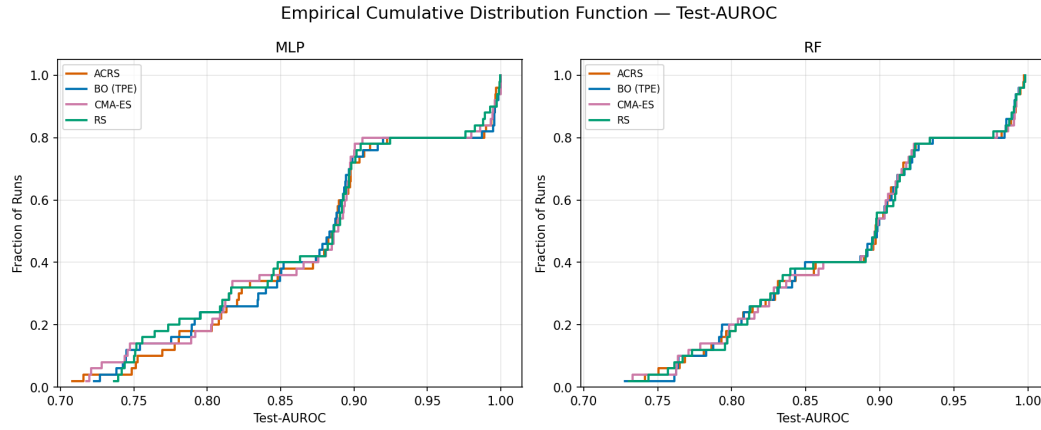


Fig. A.1.: The empirical cumulative distribution function (ECDF) of the Test-AUROC is shown in two panels, one for the MLP and one for the RF.

The four curves overlap closely across both model types, with no method's distribution extending substantially beyond the others.

A.3 Convergence Curves for All Datasets

Figures A.2 and A.3 show the full convergence behavior across all five datasets, complementing the representative excerpts in Figures 6.1 and 6.2.

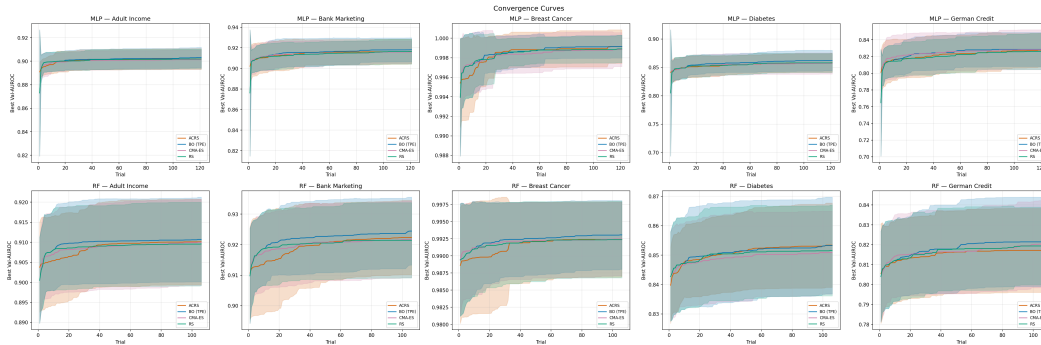


Fig. A.2.: The best Val-AUROC achieved so far is plotted against the trial number, averaged over the ten seeds, with the shaded area denoting ± 1 across-seed standard deviation. Each row corresponds to one model (MLP top, RF bottom) and each column to one dataset.

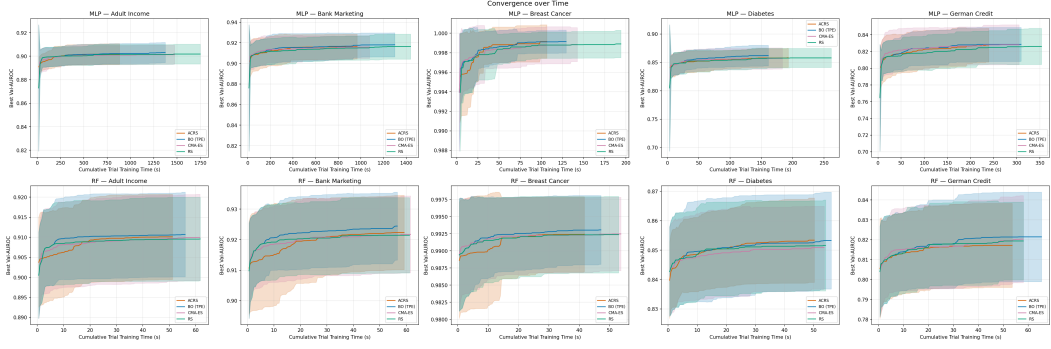


Fig. A.3.: The best Val-AUROC achieved so far is plotted against the cumulative training time, averaged over the ten seeds, with the shaded area denoting ± 1 across-seed standard deviation. Each row corresponds to one model (MLP top, RF bottom) and each column to one dataset.

A.4 Runtime Table

Table A.1 lists the exact total run duration in seconds for all method $\times$ model $\times$ dataset combinations.

Tab. A.1.: The table reports the total run duration in seconds per method, model, and dataset, given as the mean and standard deviation over the ten seeds.

Dataset	Modell	ACRS	BO (TPE)	CMA-ES	RS
Adult Income	MLP	882.8 $\pm$ 480.7	1374.6 $\pm$ 410.4	1473.4 $\pm$ 545.6	1749.0 $\pm$ 357.2
Adult Income	RF	51.2 $\pm$ 22.4	57.5 $\pm$ 14.5	61.9 $\pm$ 10.3	62.0 $\pm$ 1.9
Bank Marketing	MLP	966.1 $\pm$ 547.3	1300.3 $\pm$ 491.1	1075.7 $\pm$ 253.8	1438.6 $\pm$ 167.9
Bank Marketing	RF	59.2 $\pm$ 24.3	58.1 $\pm$ 15.2	61.9 $\pm$ 11.0	61.5 $\pm$ 1.9
Breast Cancer	MLP	106.5 $\pm$ 48.3	130.9 $\pm$ 37.8	142.9 $\pm$ 42.0	194.3 $\pm$ 30.9
Breast Cancer	RF	41.7 $\pm$ 15.8	48.6 $\pm$ 14.0	54.0 $\pm$ 8.7	53.1 $\pm$ 1.7
Diabetes	MLP	192.9 $\pm$ 156.9	162.6 $\pm$ 60.9	183.7 $\pm$ 65.2	261.6 $\pm$ 30.4
Diabetes	RF	50.2 $\pm$ 19.8	57.5 $\pm$ 13.1	53.9 $\pm$ 10.9	54.3 $\pm$ 1.3
German Credit	MLP	239.4 $\pm$ 186.6	310.9 $\pm$ 91.0	305.2 $\pm$ 69.8	353.7 $\pm$ 36.7
German Credit	RF	53.6 $\pm$ 24.2	66.8 $\pm$ 11.2	58.4 $\pm$ 13.6	58.3 $\pm$ 1.5

Colophon

This thesis was typeset with $\text{\LaTeX}2_{\epsilon}$. It uses the *Clean Thesis* style developed by Ricardo Langner. The design of the *Clean Thesis* style is inspired by user guide documents from Apple Inc.

Download the *Clean Thesis* style at <http://cleanthesis.der-ric.de/>.

Adaptations to the style of the Institute of Computer Science can be found at <https://gitlab.rlp.net/institut-fur-informatik/cleanthesis-jgu>.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe (dazu zählen auch „KI-Werkzeuge“). Sämtliche wörtlichen und sinngemäßen Übernahmen und Zitate sind kenntlich gemacht und nachgewiesen. Auch beim Einsatz von KI-Werkzeugen¹ stellt die Arbeit klar dar, in welchem Umfang diese genutzt wurden und welche Inhalte dadurch erzeugt oder beeinflusst wurden². Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferinnen und Prüfer explizit ausgeschlossen haben.

Mit Abgabe der vorliegenden Leistung übernehme ich die Verantwortung für das eingereichte Gesamtprodukt. Ich verantworte damit auch alle KI-generierten Inhalte, die ich in meine Arbeit übernommen habe. Die Richtigkeit übernommener (KI-generierter) Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen geprüft.

Ich habe die Arbeit nicht zum Erwerb eines anderen Leistungsnachweises in gleicher oder ähnlicher Form eingereicht. Von der Ordnung zur Sicherung guter wissenschaftlicher Praxis und zum Umgang mit wissenschaftlichem Fehlverhalten habe ich Kenntnis genommen.

Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte prüfungsrechtliche Konsequenzen hat und insbesondere dazu führen kann, dass die Studien- und Prüfungsleistung als mit „nicht bestanden“ bewertet wird. Die Einschreibung kann für bis zu zwei Jahre widerrufen werden, wenn Studierende zweimal oder häufiger bei Prüfungsleistungen täuschen (§ 69 Abs. 4 und 5 HochSchG).

Mainz, June 11, 2026

Dennis Diehl

¹Mit „KI-Werkzeugen“ werden computergestützte Systeme bezeichnet, die auf Basis maschinellen Lernens neue Inhalte generieren können (z. B. ChatGPT, Gemini, Claude, LLaMA, Midjourney, Stable Diffusion o. ä.).

²Die Nutzung von KI-Werkzeugen muss dann kenntlich gemacht werden, wenn Sie den Kern der Aufgabenstellung in von den Prüfern nicht anzunehmender Weise berührt: Sollte nichts anderes explizit mit den Prüferinnen und Prüfern vereinbart worden sein, so ist dies der Fall, sobald wesentliche Teile der eingereichten Arbeit (z. B. ganze Sätze des Textes, ganze Abbildungen, nicht-offensichtliche Teile von Rechnungen oder Beweisen, mehrere Zeilen von Programmcode am Stück) von solchen Werkzeugen generiert wurden und wörtlich oder in abgewandelter Form in die Arbeit übernommen wurden oder wesentliche Konzepte oder Ideen (z. B. eine Gliederung der Literatur oder ein Lösungsansatz für ein Problem) von KI-Werkzeugen generiert wurden.

Hinweis: Es wird im Falle von abweichenden Vereinbarungen mit den Prüferinnen und Prüfern empfohlen, jene im Vorfeld schriftlich festzuhalten und zusätzlich auch noch einmal in der eingereichten Arbeit zu benennen.

Use of AI tools.

AI Tool	Used for	Reason	When
Claude Sonnet 4.6	Improving phrasing and textual transitions	Enhance language quality and readability of the written text	Throughout the writing process
Claude Opus 4.7	Error analysis and bug fixing in the source code	Debugging and ensuring correctness of the implementation	When the code failed to run or produced incorrect results

